

MULTI-LAYER PERCEPTRON

CSE 368 Artificial Intelligence

Based on: Varun Chandola's CSE 574 Course, Stanford cs231n, Berkley CS 188

ML

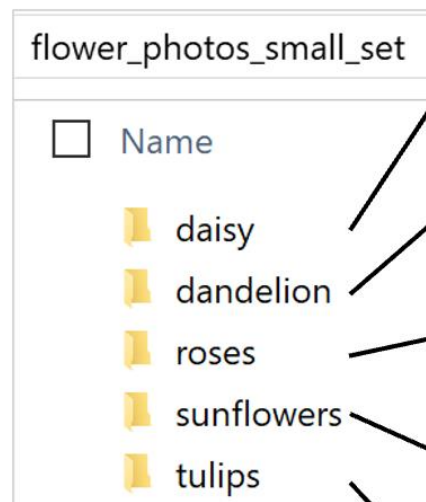
Dataset



Position	Experience	Skill	Country	City	Salary (\$)
Developer	0	1	USA	New York	103100
Developer	1	1	USA	New York	104900
Developer	2	1	USA	New York	106800
Developer	3	1	USA	New York	108700
Developer	4	1	USA	New York	110400
Developer	5	1	USA	New York	112300
Developer	6	1	USA	New York	114200
Developer	7	1	USA	New York	116100
Developer	8	1	USA	New York	117800
Developer	9	1	USA	New York	119700
Developer	10	1	USA	New York	121600

Dataset

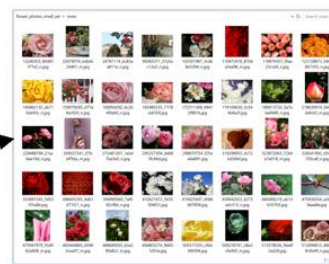
date	time	Inputv1	Inputv2	Inputv3	Inputv4	Inputv5	Inputv6	output
8/29/2018	19:50:00	1	0	0	1	0	0	1
8/29/2018	19:55:00	1	0	0	1	0	0	0
8/29/2018	20:00:00	1	0	0	1	0	0	1
8/29/2018	20:05:00	1	1	1	0	0	0	1
8/29/2018	20:10:00	1	1	1	0	0	0	1
8/29/2018	20:15:00	1	1	0	1	0	0	1
8/29/2018	20:20:00	1	1	0	1	1	0	0
8/29/2018	20:25:00	1	0	0	1	1	0	1
8/29/2018	20:30:00	1	0	0	1	1	0	1
8/29/2018	20:35:00	0	0	0	1	0	0	1
8/29/2018	20:40:00	1	0	0	1	1	0	1
8/29/2018	20:45:00	0	0	0	1	0	0	0



40 image files



40 image files



40 image files

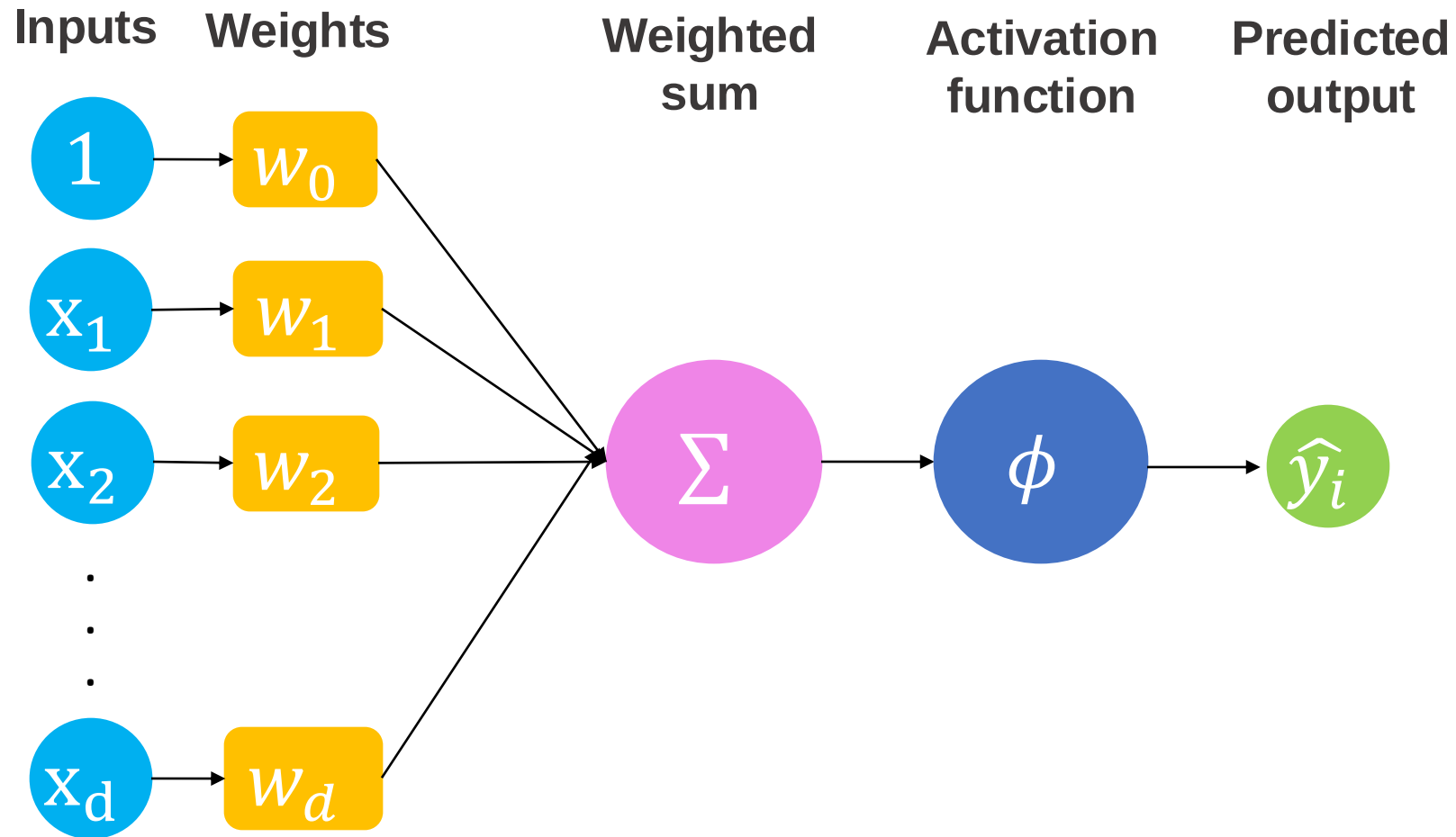


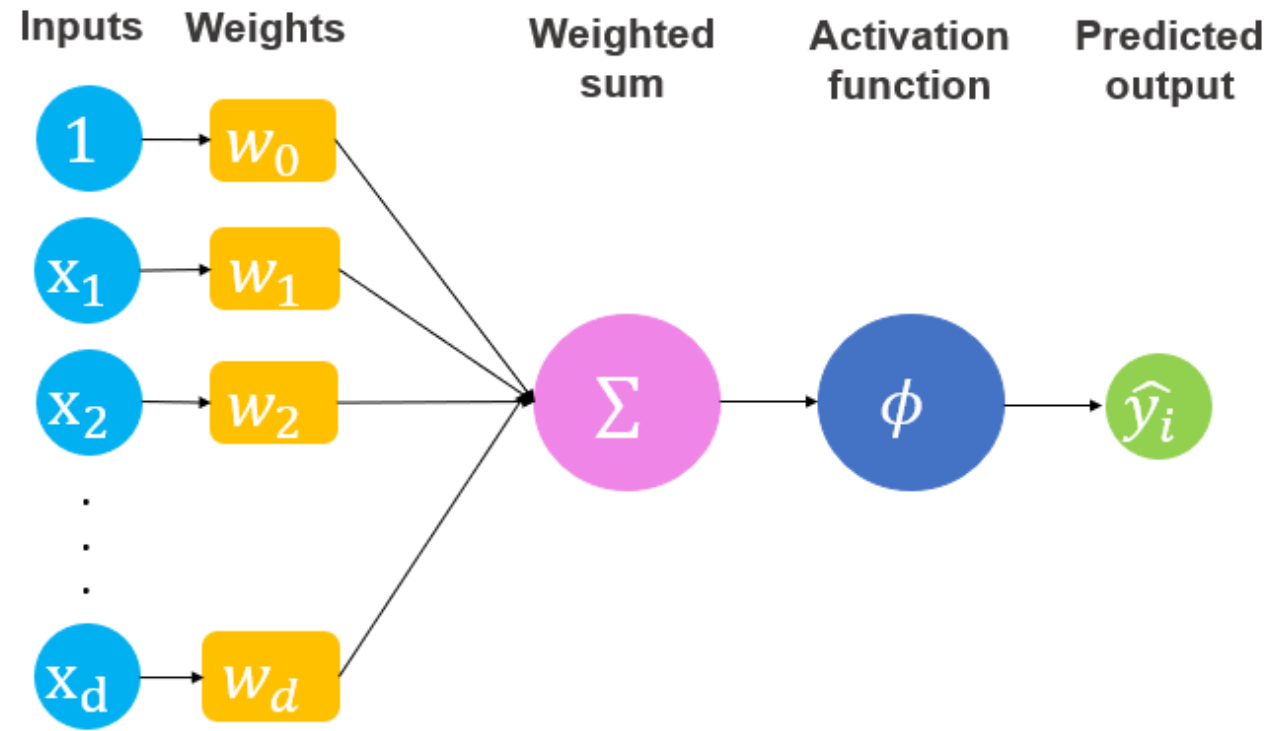
40 image files



40 image files

Perceptron

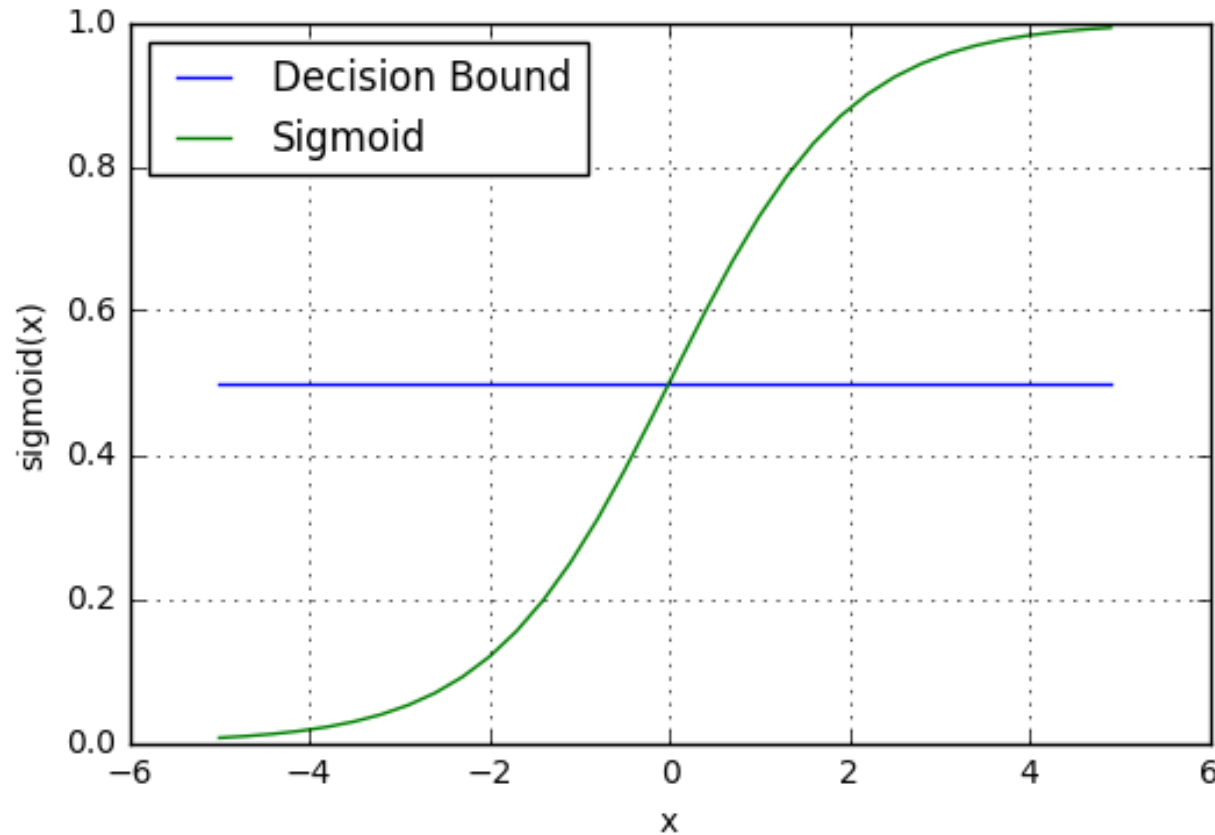




Decision Rule

$$\hat{y}_i = f_{\mathbf{w}}(x) = \text{sign} \left(w_0 + \sum_{j=1}^d w_j x_j \right) = \begin{cases} +1 & \text{if } w_0 + \mathbf{w}^T \mathbf{x}_i \geq 0 \\ -1 & \text{if } w_0 + \mathbf{w}^T \mathbf{x}_i < 0 \end{cases}$$

Sigmoid (Logistic)



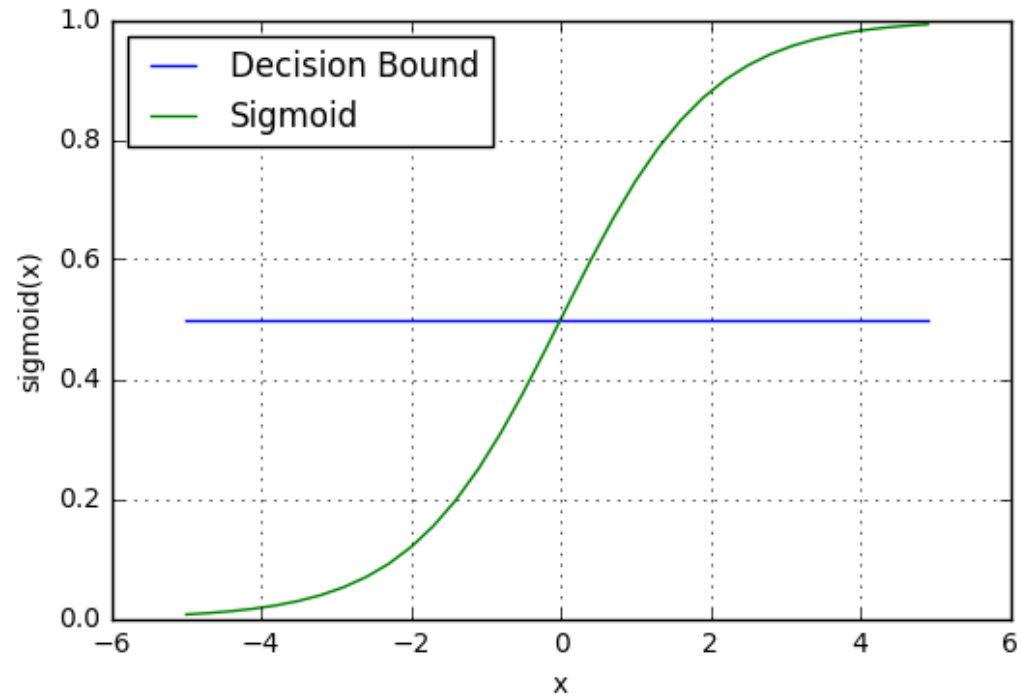
$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

Range : (0 to 1) probability estimate

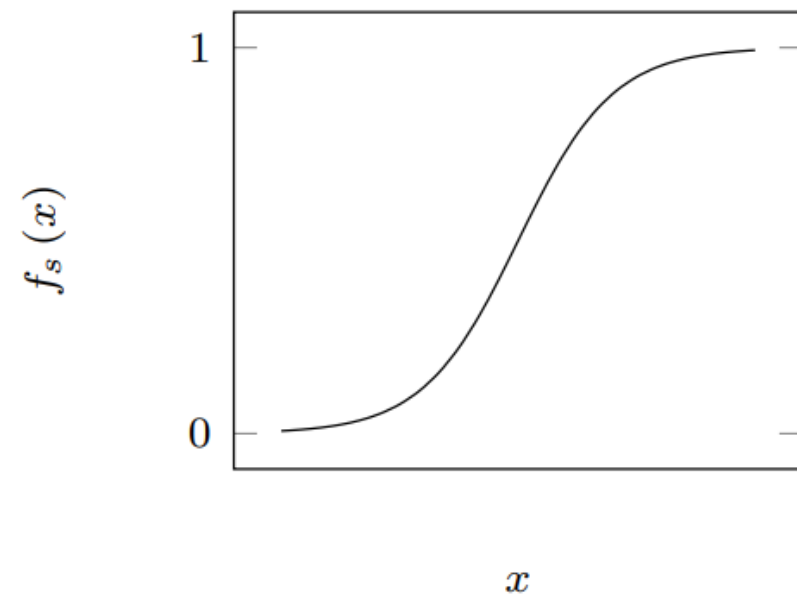
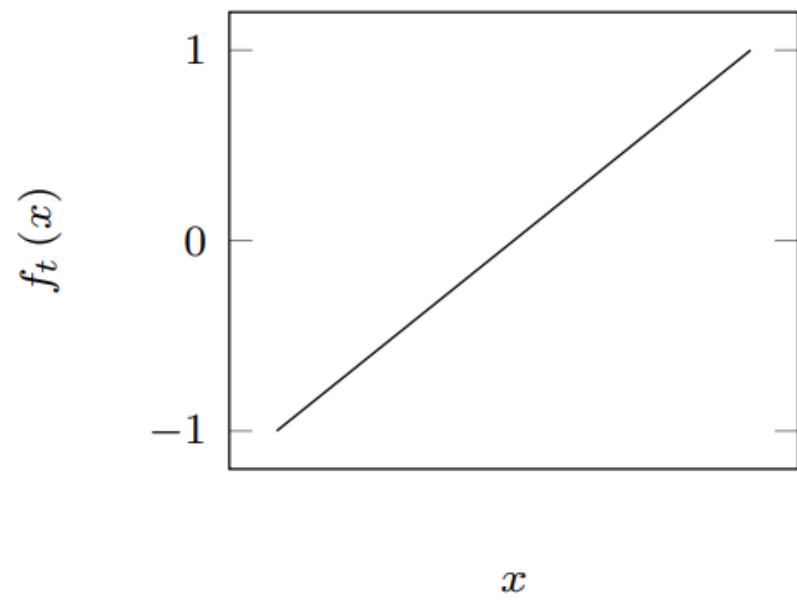
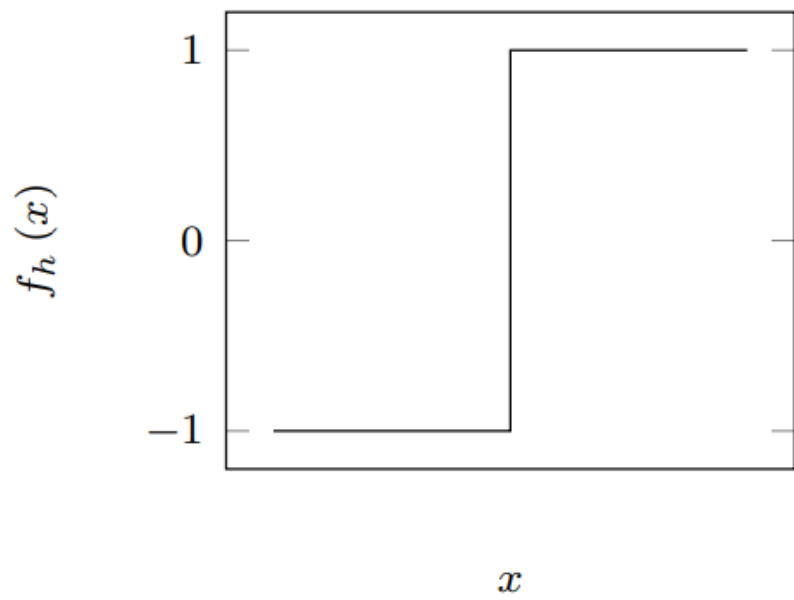
- The output is not zero centered
- e = base of natural log
- $\text{Exp}()$ is a relatively expensive operation
- Small active region
- The function is **differentiable**

```
def sigmoid(x):  
    return 1.0 / (1 + np.exp(-x))
```

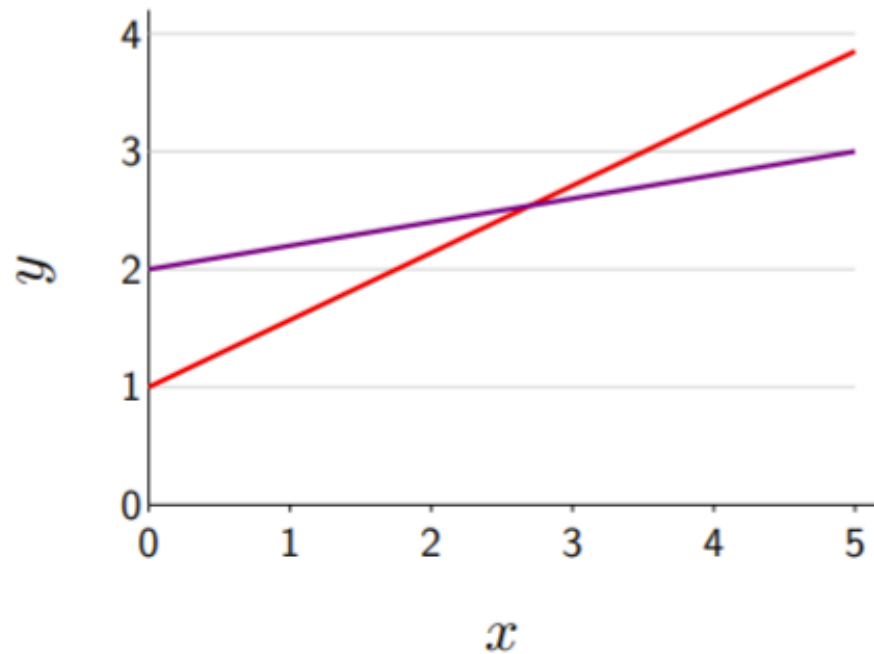
Sigmoid (Logistic)



$$f_{\mathbf{w}}(x) = \sigma(\mathbf{w} * \phi(x)) = \begin{cases} +1 & \text{if } \frac{1}{1 + e^{-x}} \geq 0.5 \\ -1 & \text{if } \frac{1}{1 + e^{-x}} < 0.5 \end{cases}$$



Linear regression

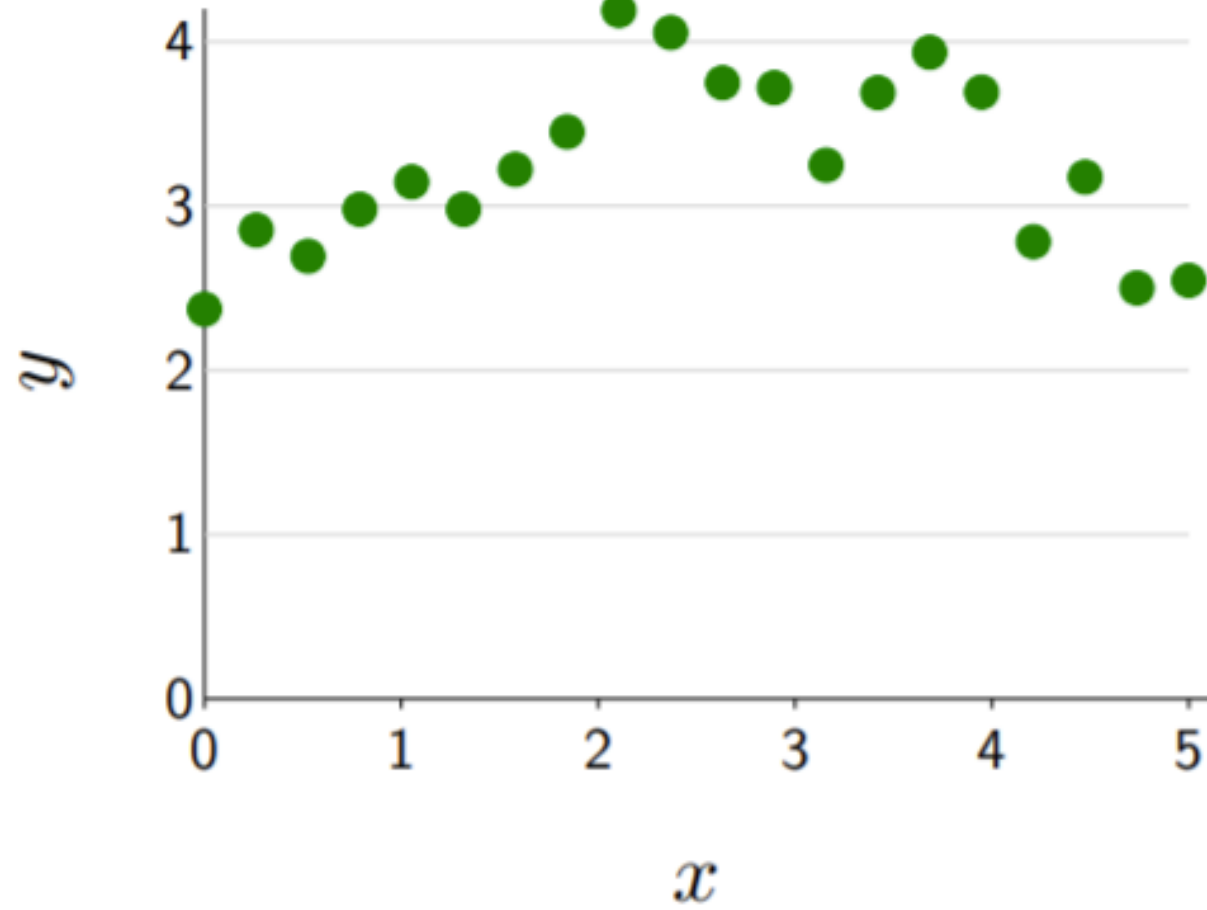


$$f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)$$

$$\phi(x) = [1, x]$$

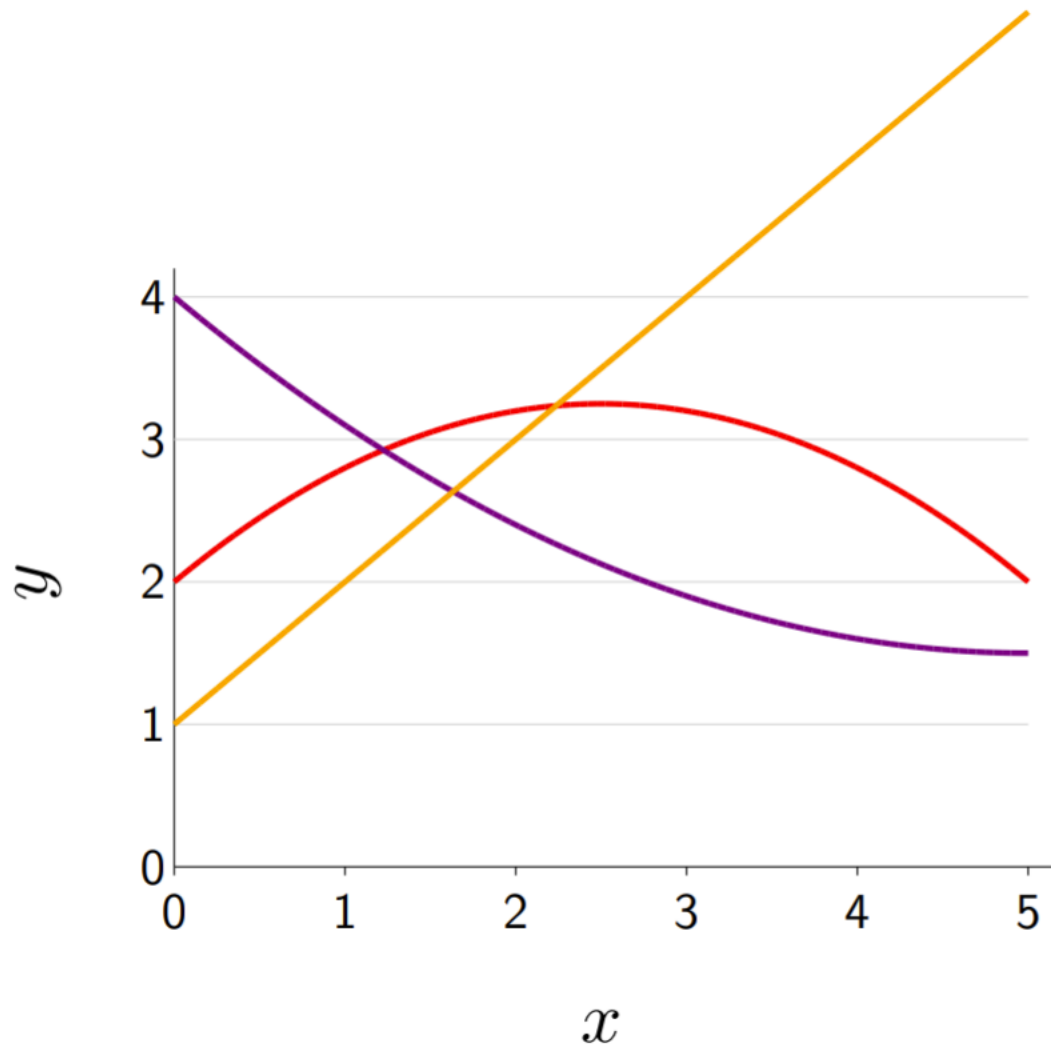
$$f(x) = [1, 0.57] \cdot \phi(x)$$

$$f(x) = [2, 0.2] \cdot \phi(x)$$



How do we fit a non-linear predictor?

Non-linear (quadratic) predictors



$$f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x)$$

$$\phi(x) = [1, x, x^2]$$

change
feature
extractor

Example: $\phi(3) = [1, 3, 9]$

$$f(x) = [2, 1, -0.2] \cdot \phi(x)$$

$$f(x) = [4, -1, 0.1] \cdot \phi(x)$$

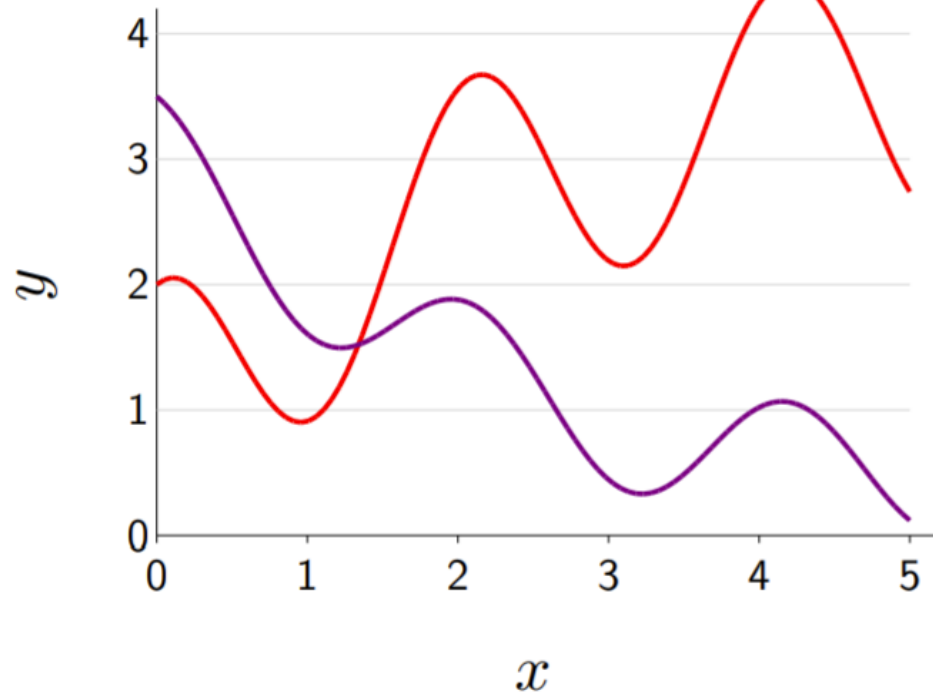
$$f(x) = [1, 1, 0] \cdot \phi(x)$$

$$\mathcal{F} = \{f_{\mathbf{w}}(x) = \mathbf{w} \cdot \phi(x) : \mathbf{w} \in \mathbb{R}^3\}$$

How to learn non-linear surfaces?

Add another layer – **Multi-Layer Perceptron (Neural Network)**

Non-linear neural networks



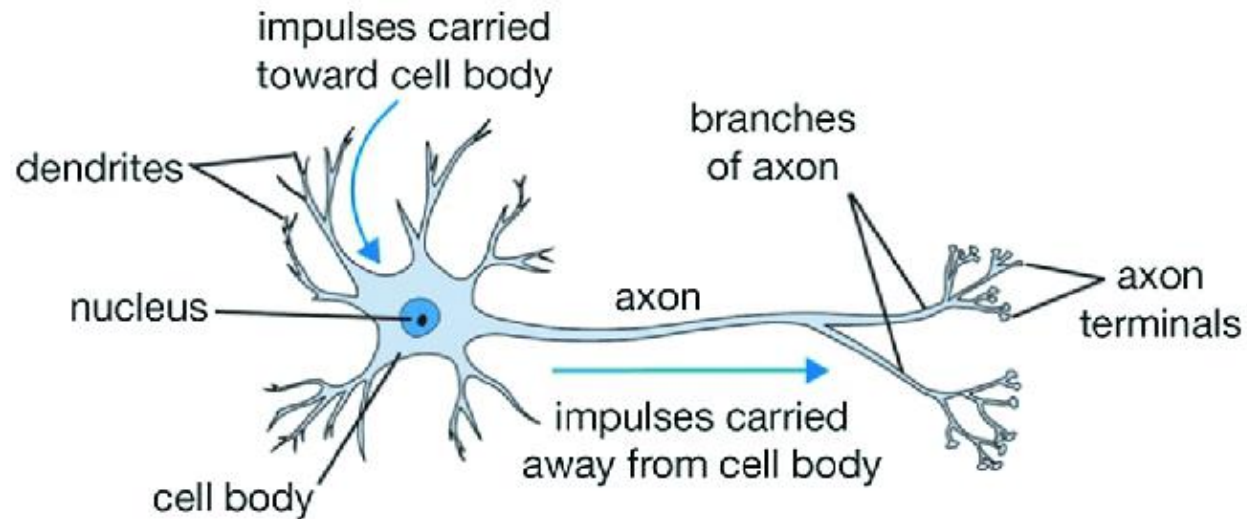
$$f_{\mathbf{w}}(x) = \mathbf{w} \cdot \sigma(\mathbf{V}\phi(x))$$

$$\phi(x) = [1, x]$$



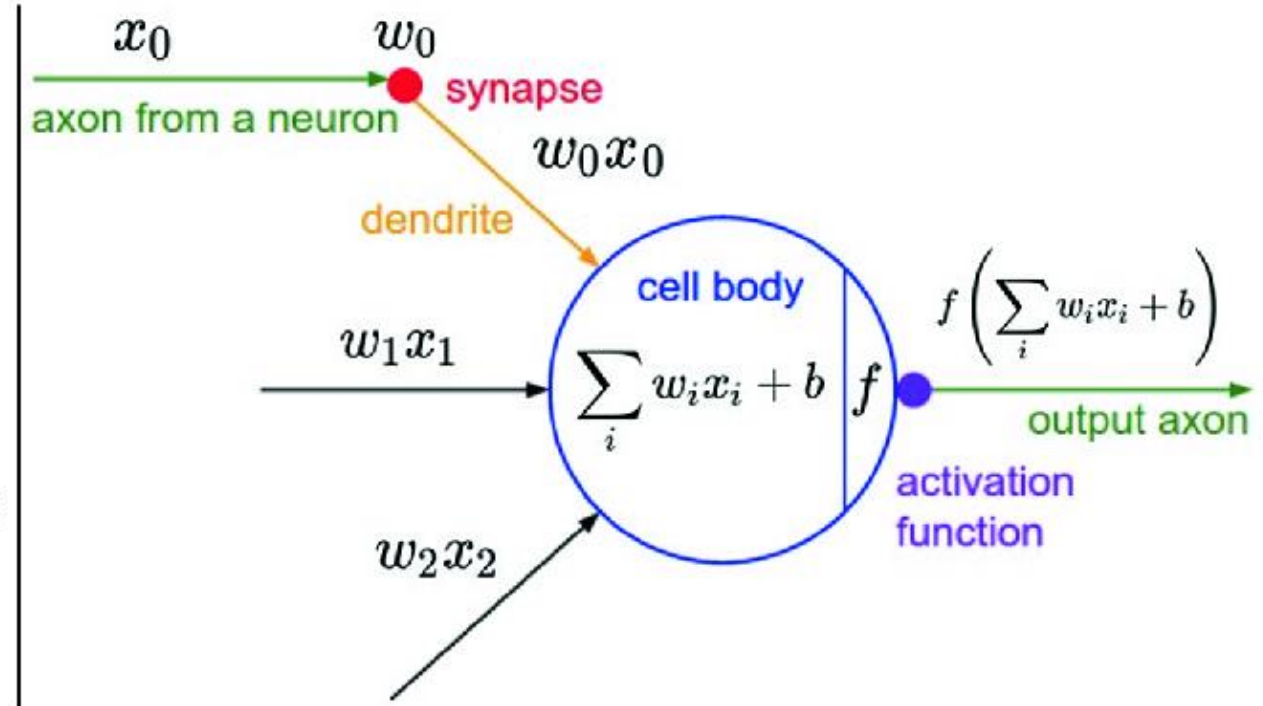
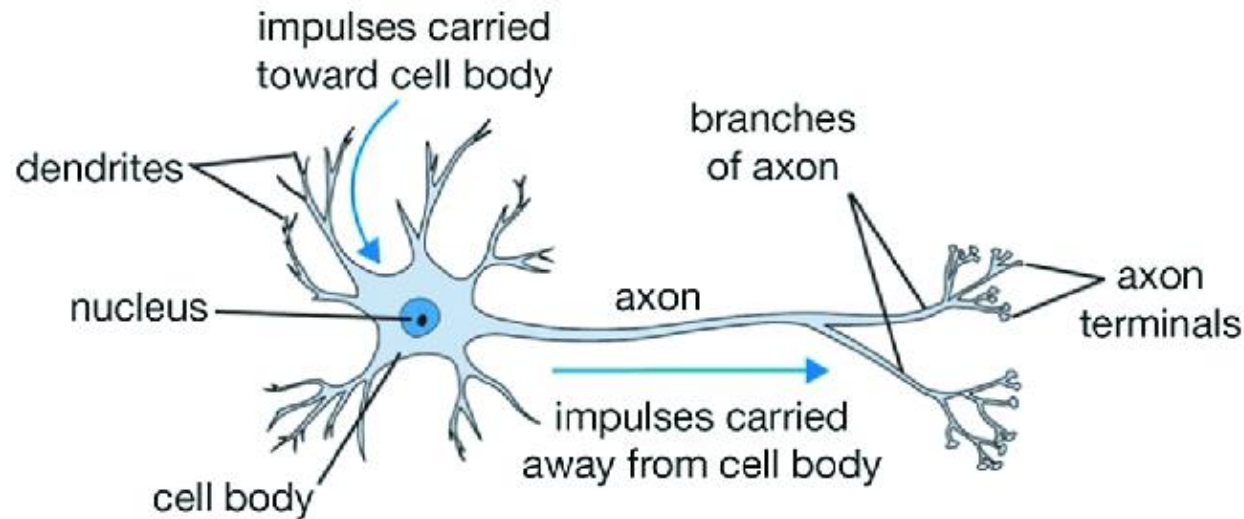
MULTI-LAYER PERCEPTRON

Biological Neuron & Mathematical Model



[Source](#)

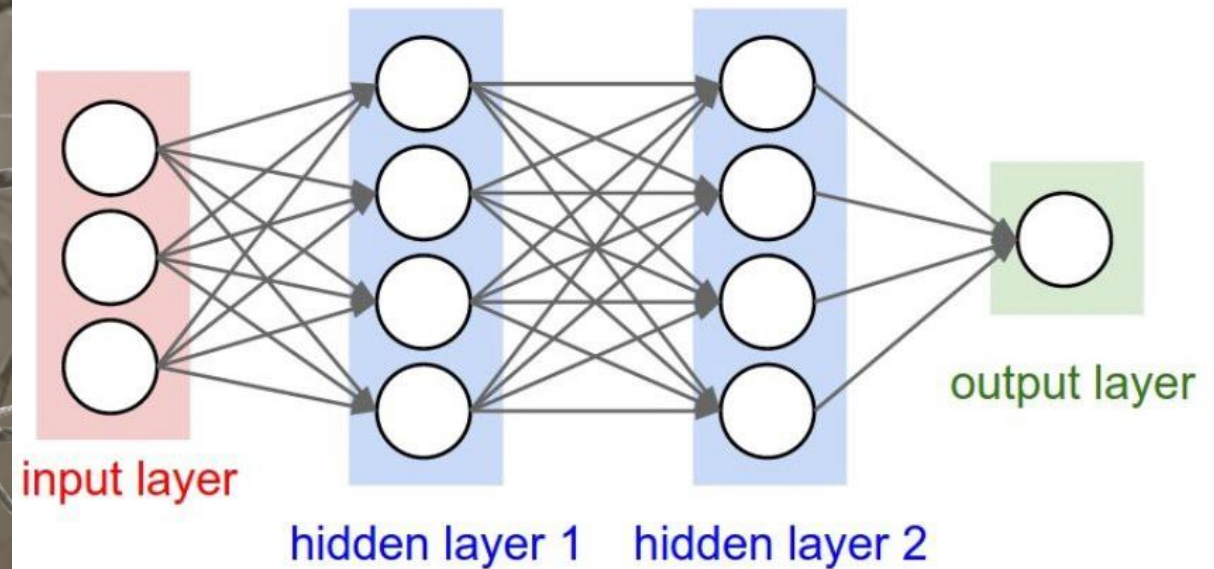
Biological Neuron & Mathematical Model



Biological Neurons:
Complex connectivity patterns



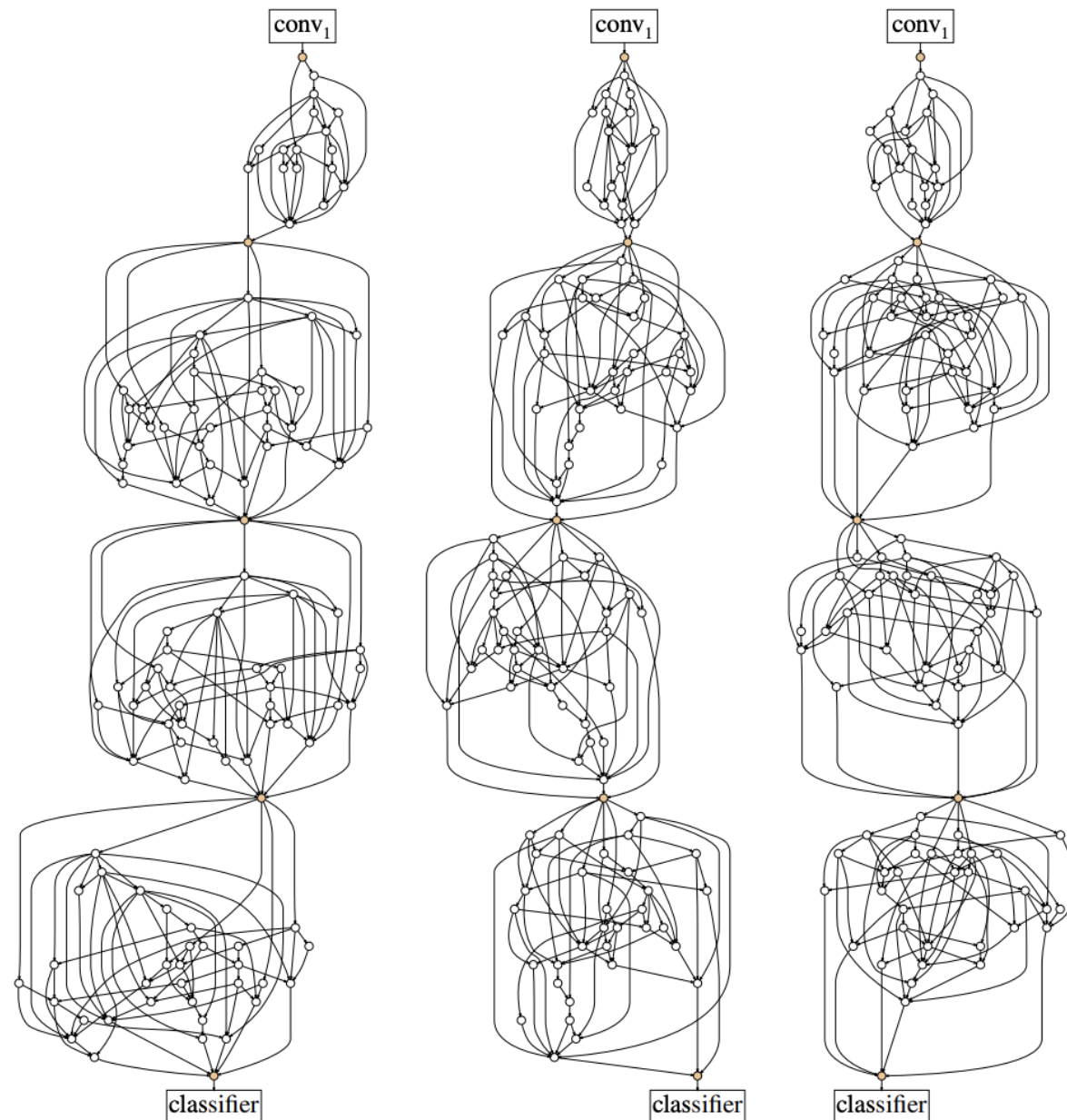
Neurons in a neural network:
Organized into regular layers for computational efficiency



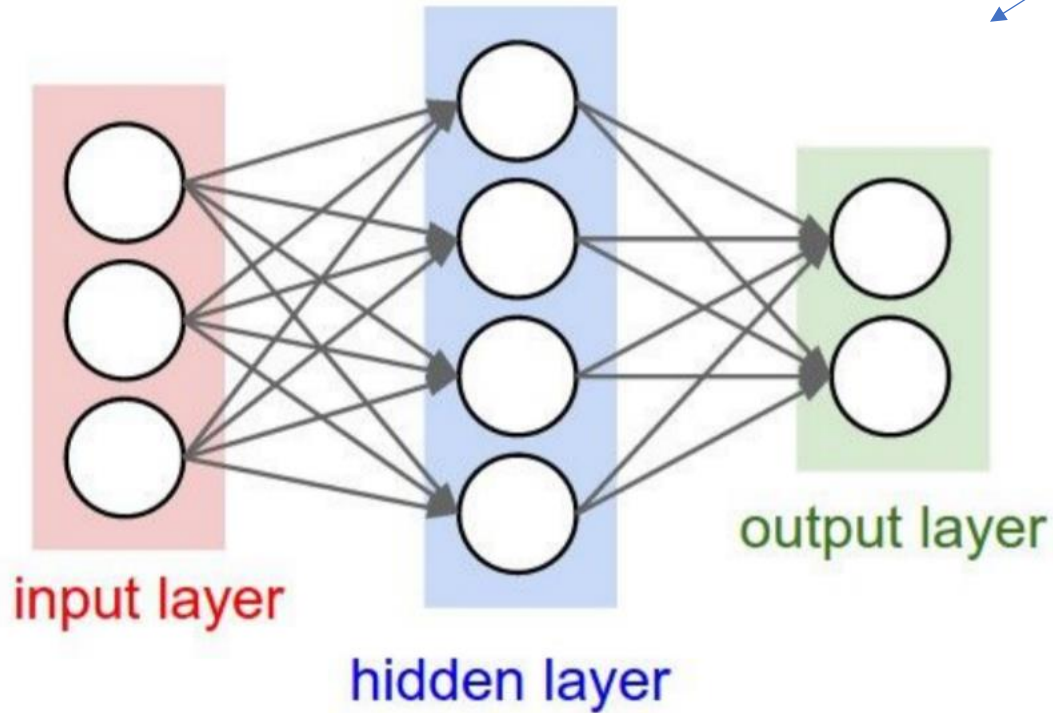
Biological Neurons: Complex connectivity patterns



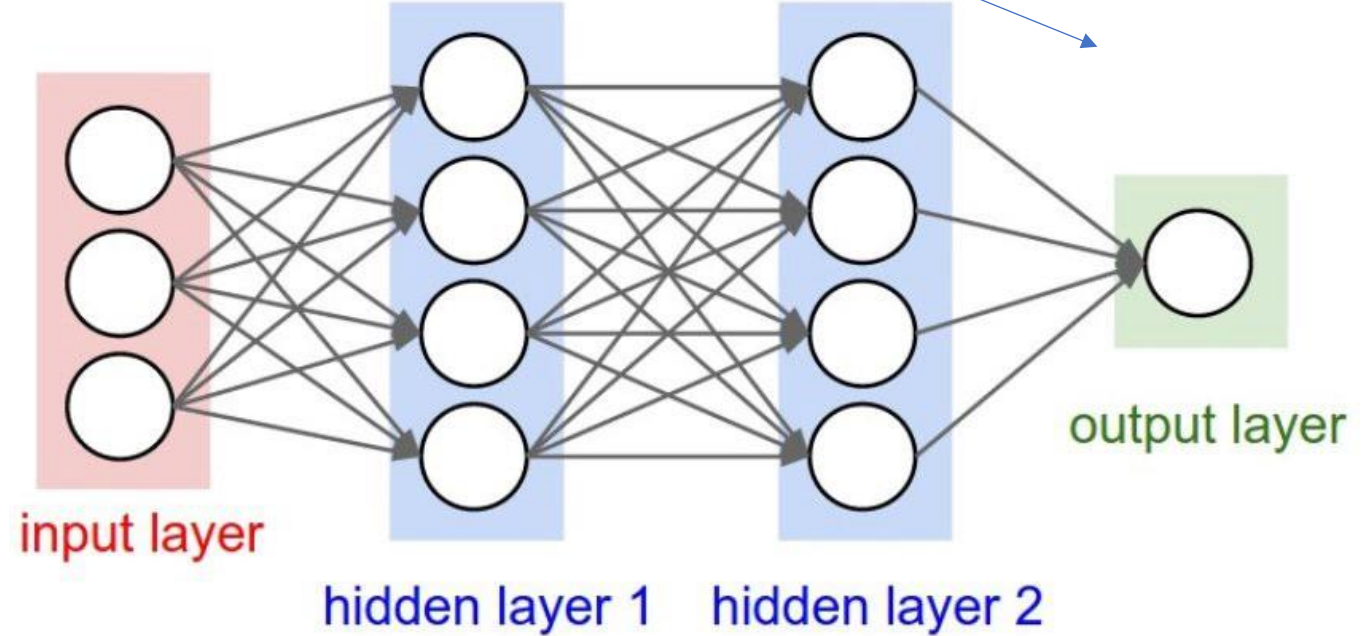
Randomly wired neural nets



Fully-connected (FC) layers



2-layer Neural Net
or
1-hidden-layer Neural Net



3-layer Neural Net
or
2-hidden-layer Neural Net

Feed Forward Neural Networks

- D input nodes
(excluding bias)
 - M hidden nodes
(excluding bias)
 - K output nodes
-
- At hidden nodes: $w_j, 1 \leq j \leq M, w_j \in \mathbb{R}^{D+1}$
 - At output nodes: $w_l, 1 \leq l \leq K, w_l \in \mathbb{R}^{M+1}$

Learning Weights of the Multi-layer Perceptron

Assume that the network structure is predetermined (number of hidden nodes and interconnections)

Objective function for N training examples:

$$J = \sum_{i=1}^N J_i = \frac{1}{2} \sum_{i=1}^N \sum_{l=1}^K (y_{il} - o_{il})^2$$

There can be other objective (or loss) functions (e.g., logistic-loss)

y_{il} - Target value associated with l class for input ($\mathbf{x}_i$)

$y_{il} = 1$ when k is true class for $\mathbf{x}_i$, and 0 otherwise

o_{il} - Predicted output value at l output node for $\mathbf{x}_i$

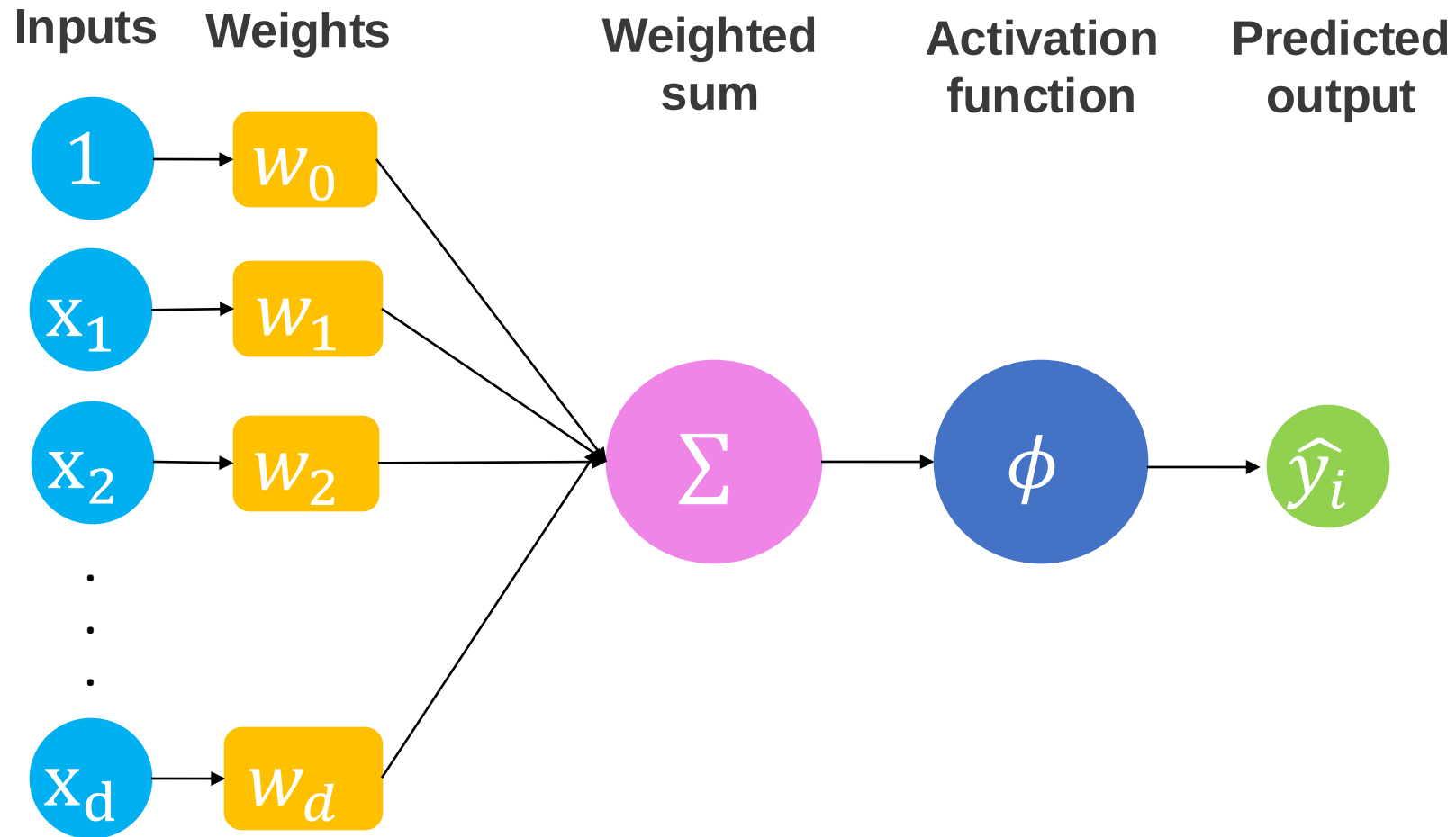
What are we learning?

Weight vectors for all output and hidden nodes that minimize J



FORWARD PASS

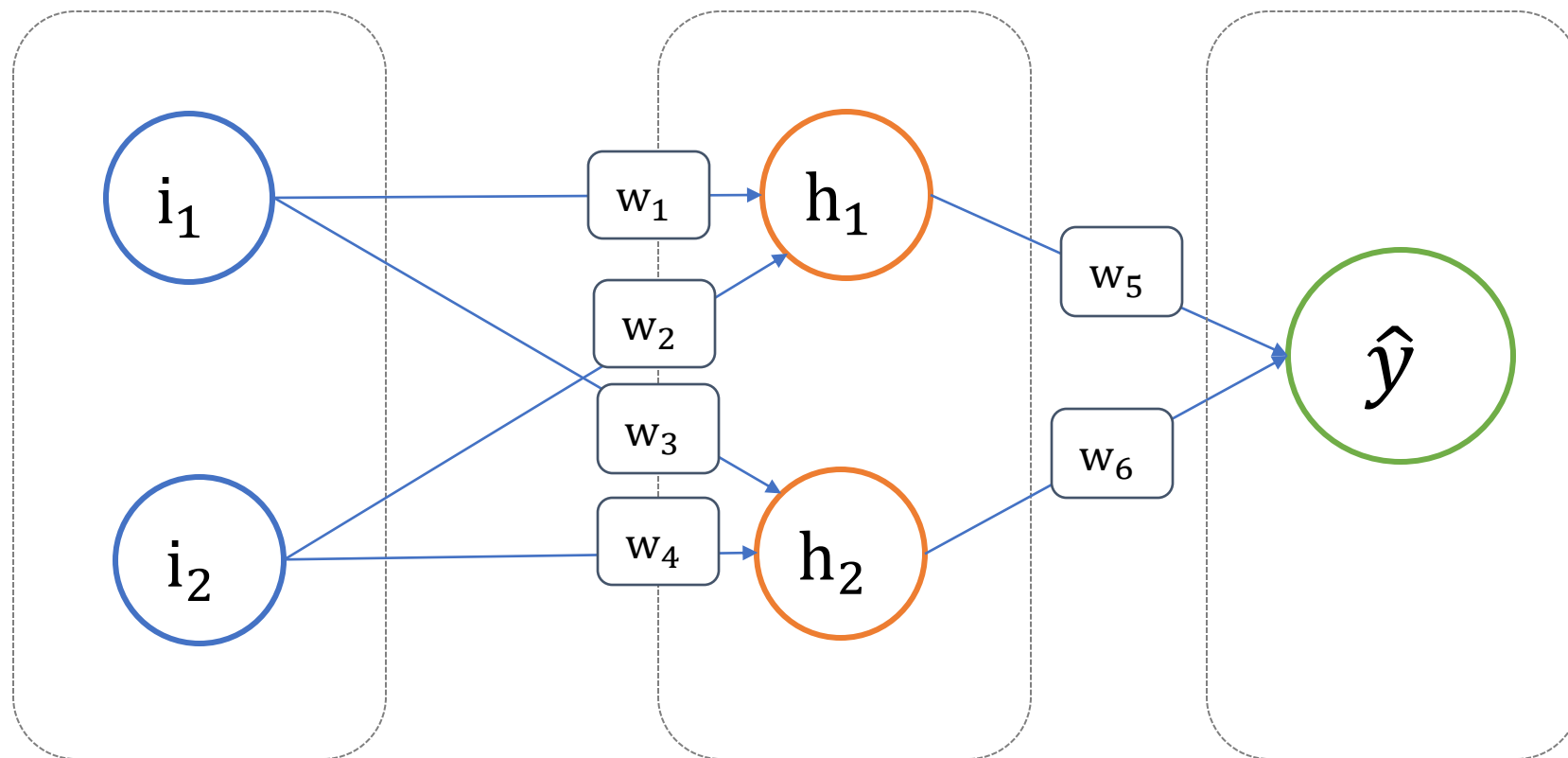
Perceptron



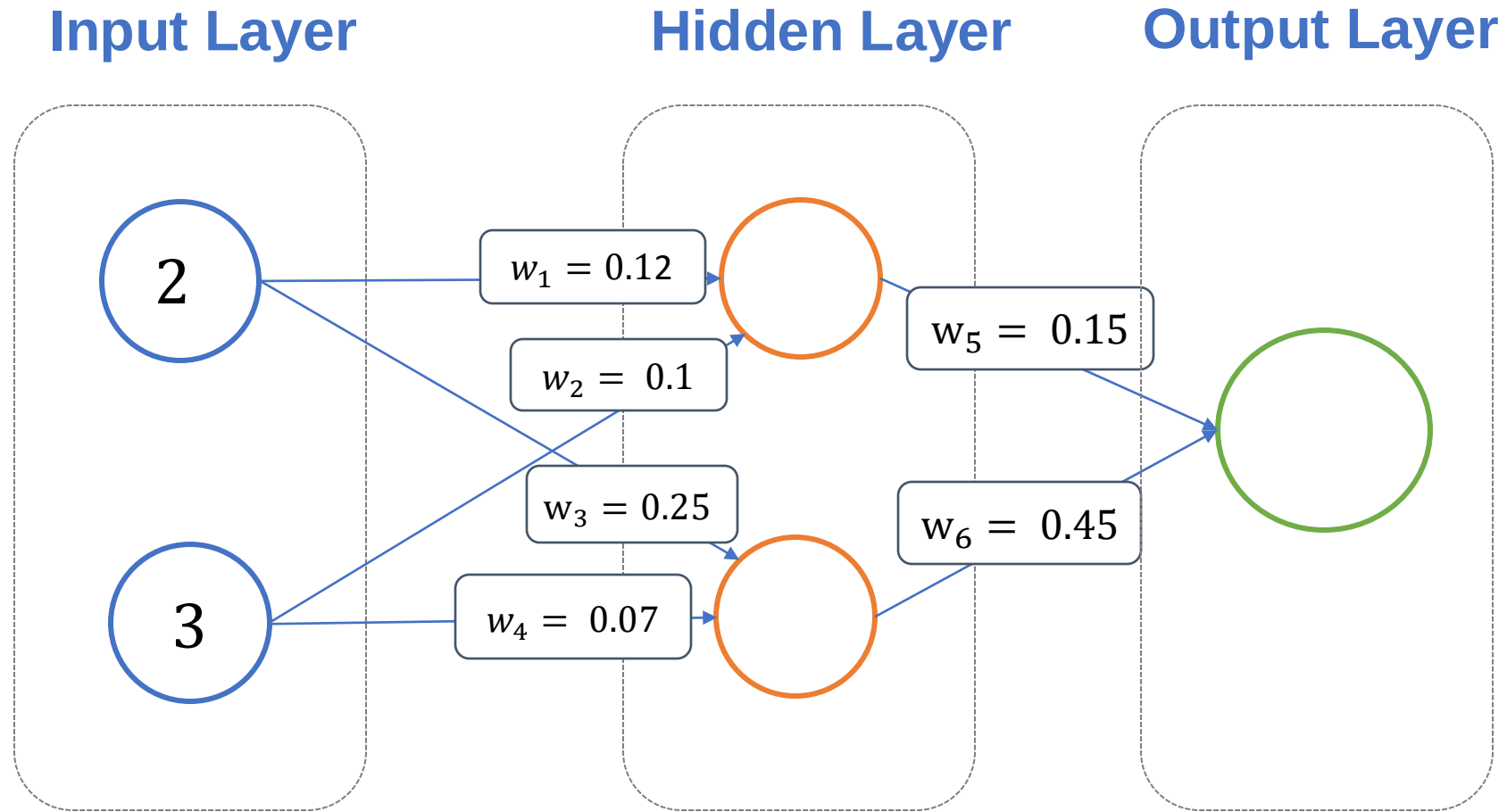
Input Layer

Hidden Layer

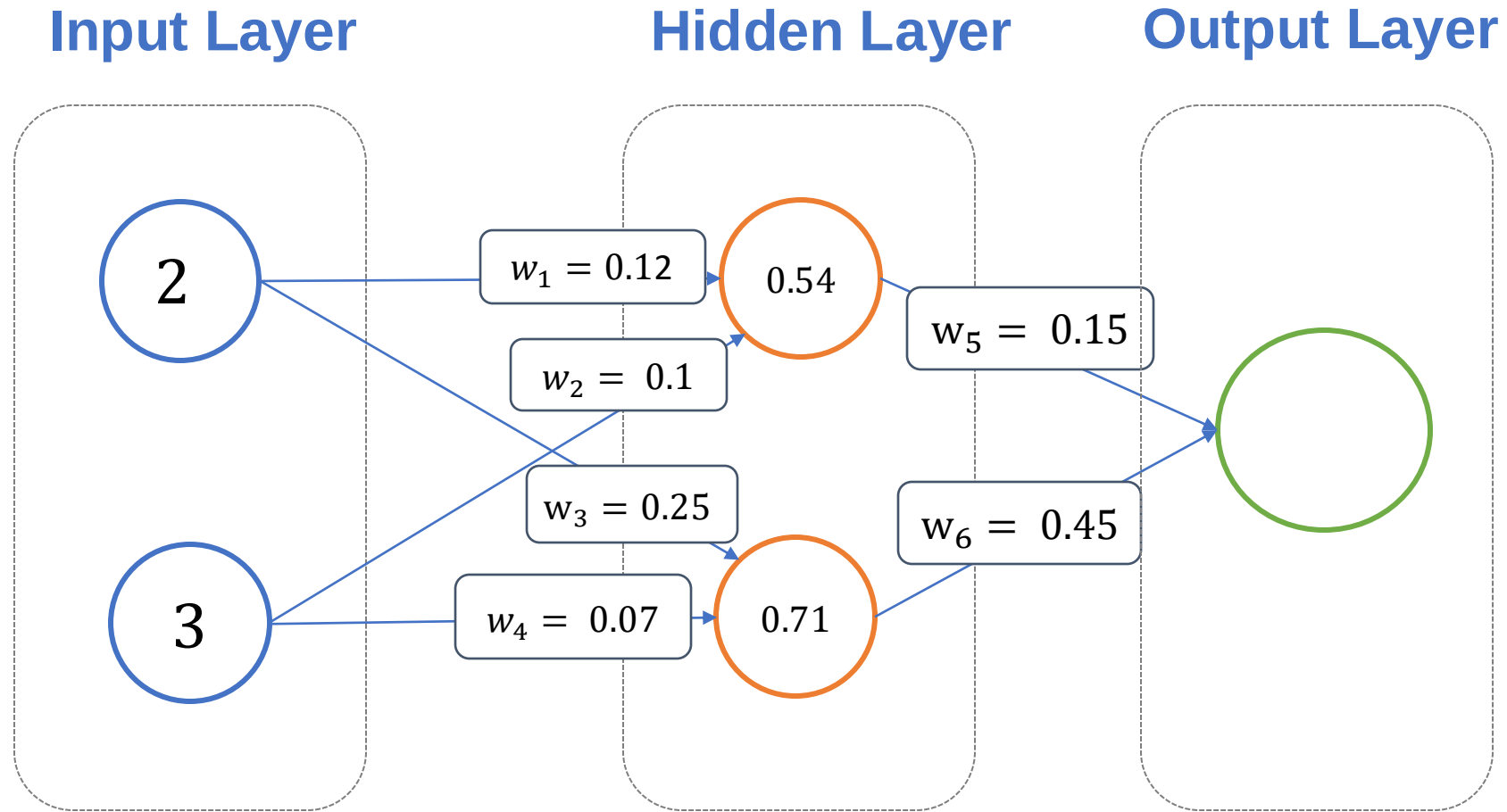
Output Layer



Forward Pass

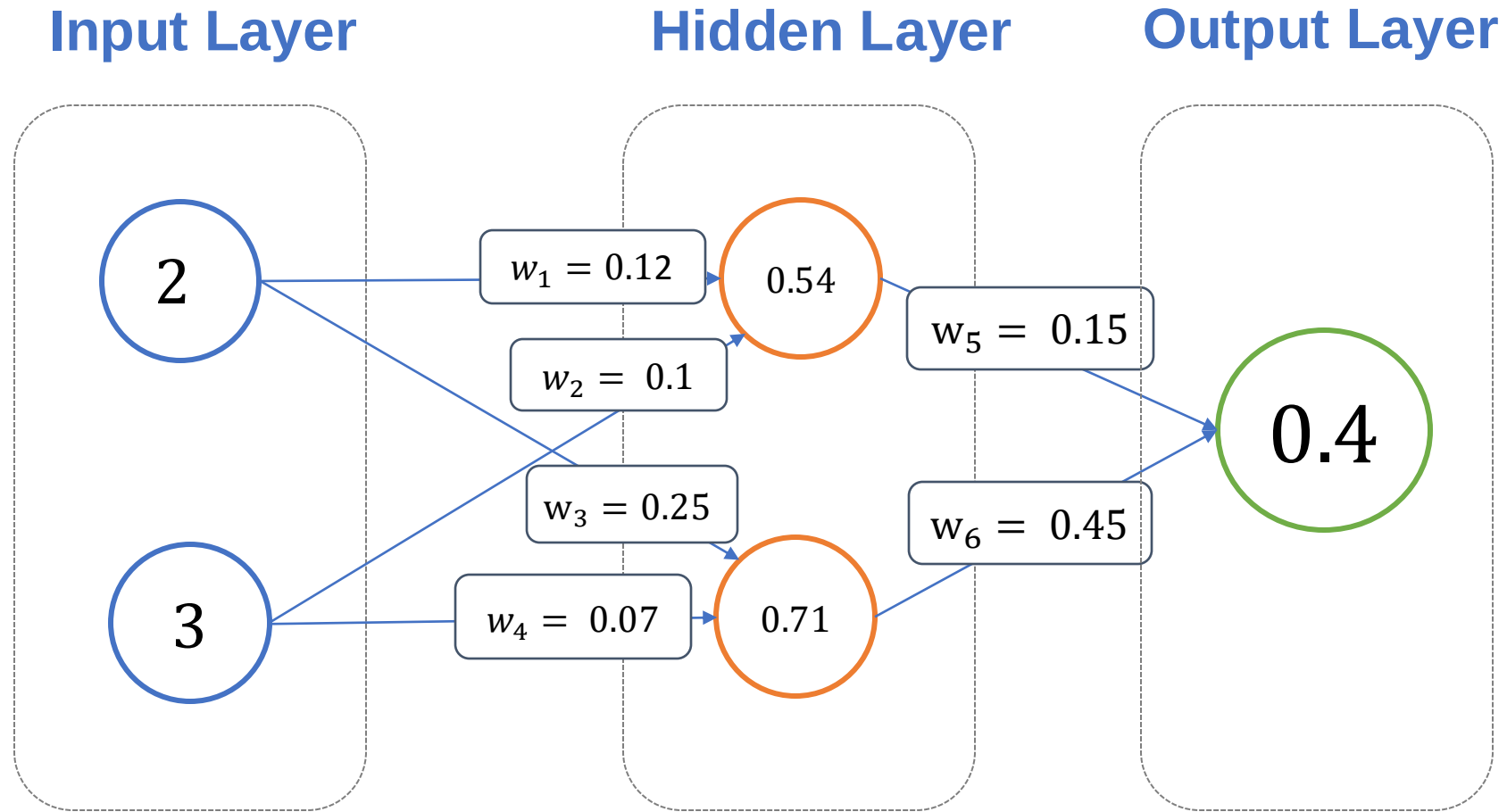


Forward Pass



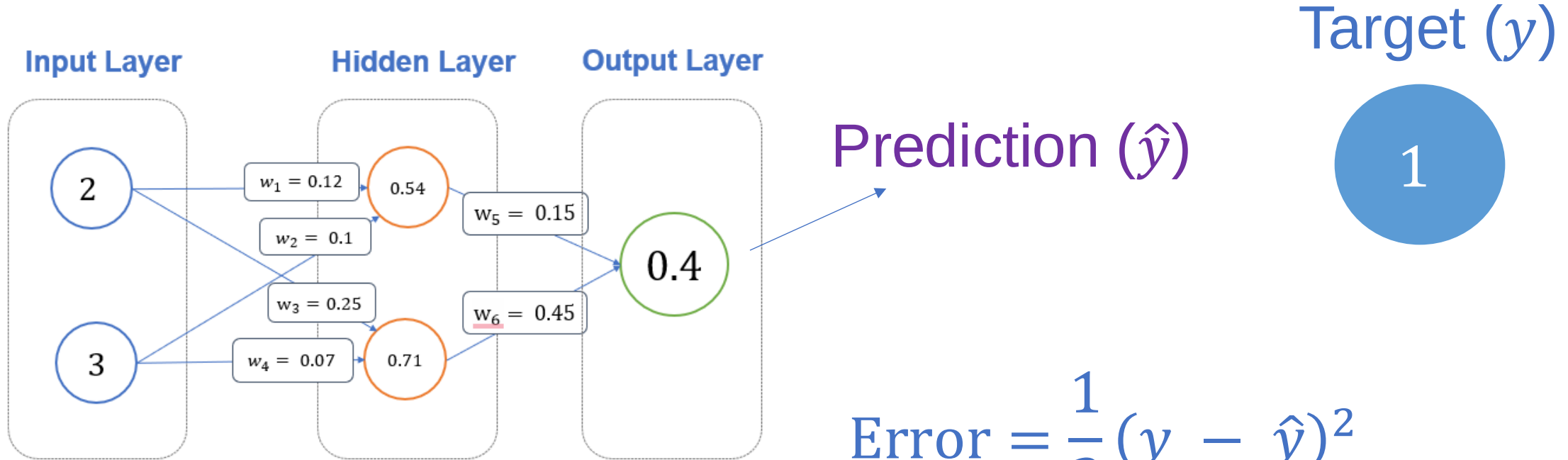
$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.12 & 0.25 \\ 0.1 & 0.07 \end{bmatrix} = \begin{bmatrix} 0.54 & 0.71 \end{bmatrix}$$

Forward Pass



$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.12 & 0.25 \\ 0.1 & 0.07 \end{bmatrix} = \begin{bmatrix} 0.54 & 0.71 \end{bmatrix} \cdot \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix} = \begin{bmatrix} 0.4 \end{bmatrix}$$

Calculating Error



$$\text{Error} = \frac{1}{2} (y - \hat{y})^2$$

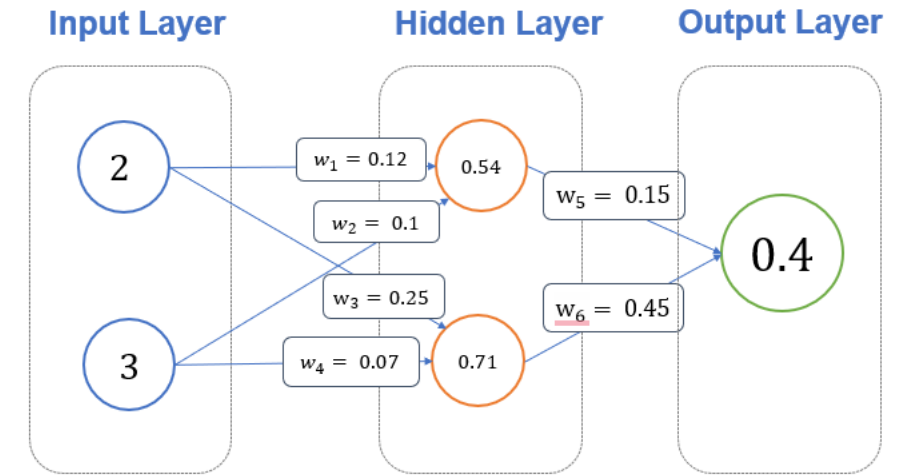
$$\text{Error} = \frac{1}{2} (1 - 0.4)^2 = 0.18$$

Reducing Error

prediction = $\hat{y}$

$$\hat{y} = (h_1)w_5 + (h_2)w_6$$

$$\hat{y} = (i_1w_1 + i_2w_3)w_5 + (i_1w_2 + i_2w_4)w_6$$



To change the prediction value,
we need to change weights

How to change/update the weights value so that
the error is reduced?

Backpropagation

A scenic view of a park with a paved path, trees, and a building in the background. The path is on the left, leading into a grassy area with many yellow dandelions. Several trees are scattered across the field, and a brick building is visible in the distance under a cloudy sky.

BACKPROPAGATION

Backpropagation

- Backpropagation (**backward propagation of errors**) is a mechanism used to update the weights using gradient descent
- It calculates the gradient of the error function with respect to the neural network's weights
- The calculation proceeds backwards through the network
- To find a local minimum of a function using gradient descent, we take steps proportional to the negative of the gradient of the function at the current point

$$w_i = w_i - \alpha \left(\frac{\partial Error}{\partial w_i} \right)$$

The Backpropagation Algorithm

1. Initialize all weights to *small values*
2. For each training example, $\langle x, y \rangle$:
 - **Propagate input forward** through the network
 - **Propagate errors backward** through the network

Backpropagation Algorithm

Gradient Descent

- Move in the opposite direction of the gradient of the objective function
- $-\eta \nabla J$

$$\nabla J = \sum_{i=1}^N \nabla J_i$$

Backpropagation Algorithm

Gradient Descent

- Move in the opposite direction of the gradient of the objective function
- $-\eta \nabla J$

$$\nabla J = \sum_{i=1}^N \nabla J_i$$

What is the gradient computed with respect to?

Backpropagation Algorithm

What is the gradient computed with respect to?

Weight vectors - M at hidden nodes and K at output nodes

$$\begin{aligned} &w_j \ (j = 1 \dots M) \\ &w_l \ (l = 1 \dots K) \\ \mathbf{w}_j &\leftarrow \mathbf{w}_j - \eta \frac{\partial J}{\partial \mathbf{w}_j} = \mathbf{w}_j - \eta \sum_{i=1}^N \frac{\partial J_i}{\partial \mathbf{w}_j} \end{aligned}$$

$$\mathbf{w}_l \leftarrow \mathbf{w}_l - \eta \frac{\partial J}{\partial \mathbf{w}_l} = \mathbf{w}_l - \eta \sum_{l=1}^N \frac{\partial J_l}{\partial \mathbf{w}_l}$$

$$\nabla J_i = \begin{bmatrix} \frac{\partial J_i}{\partial \mathbf{w}_1} \\ \frac{\partial J_i}{\partial \mathbf{w}_2} \\ \vdots \\ \frac{\partial J_i}{\partial \mathbf{w}_{m+k}} \end{bmatrix}$$

Final Algorithm

While not converged:

Move forward to compute outputs at hidden and output nodes

Move backward to propagate errors back

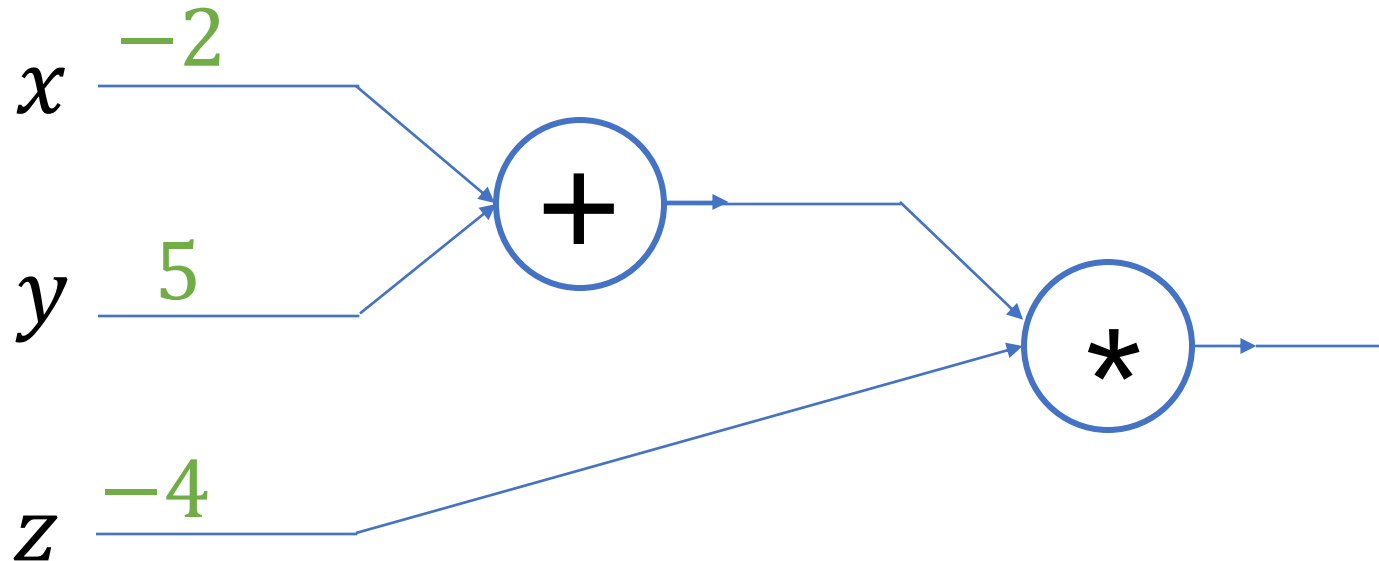
- Compute δ errors at output nodes (δ_l)
- Compute δ errors at hidden nodes (δ_j)

Update all weights according to weight update equations

Backpropagation: Simple example

$$f(x, y, z) = (x + y)z$$

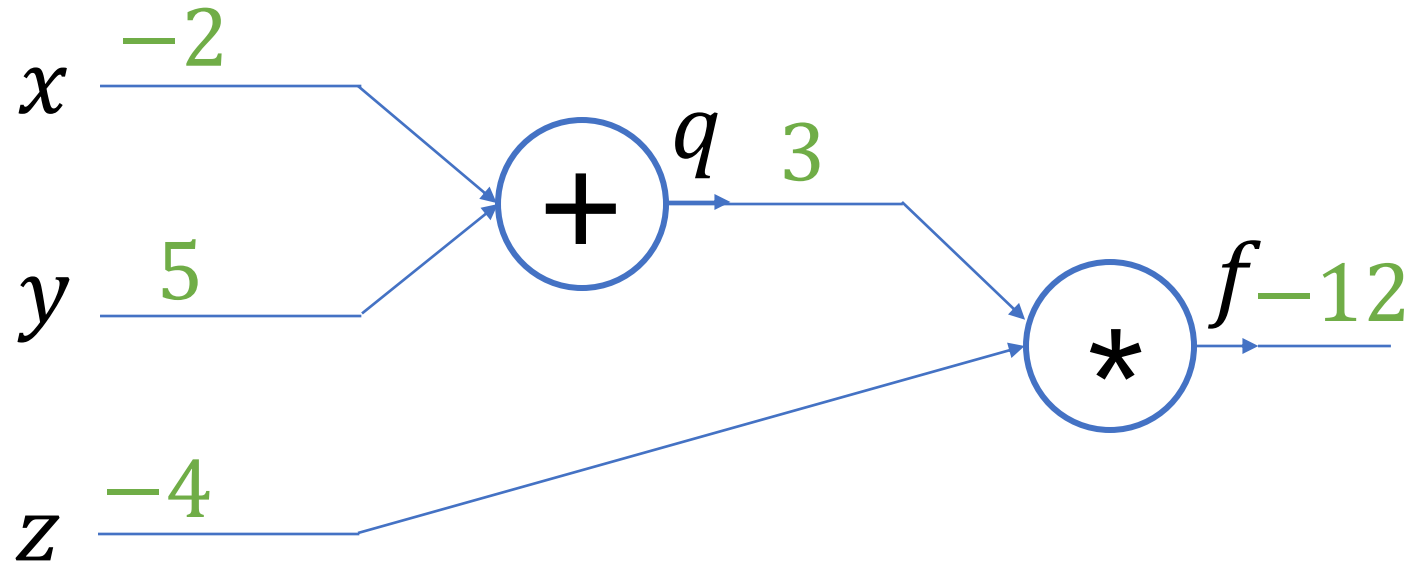
$$x = -2, y = 5, z = -4$$



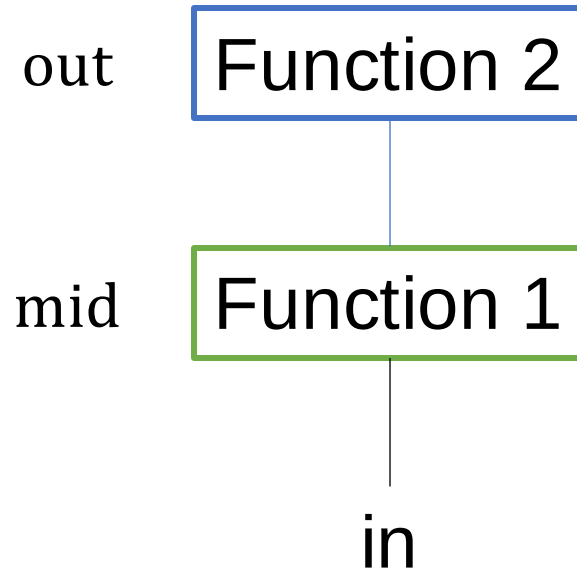
Backpropagation: Simple example

$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$



Composing functions



Chain rule:

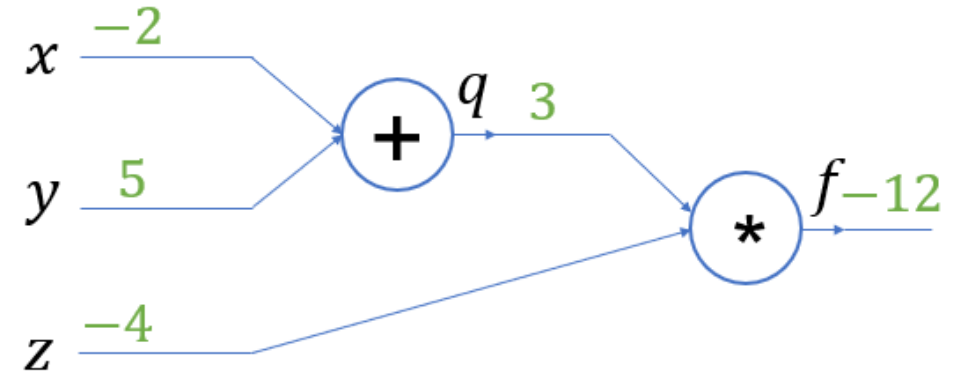
$$\frac{\partial \text{out}}{\partial \text{in}} = \frac{\partial \text{out}}{\partial \text{mid}} \frac{\partial \text{mid}}{\partial \text{in}}$$

Backpropagation: Simple example

$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} =$$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Backpropagation: Simple example

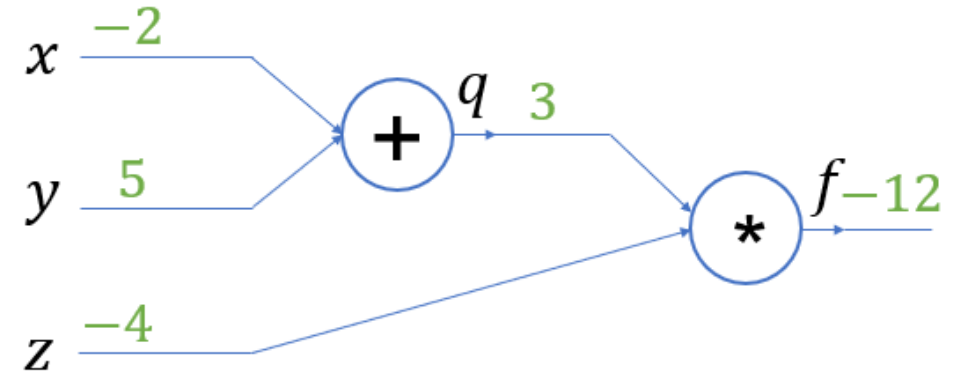
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: Simple example

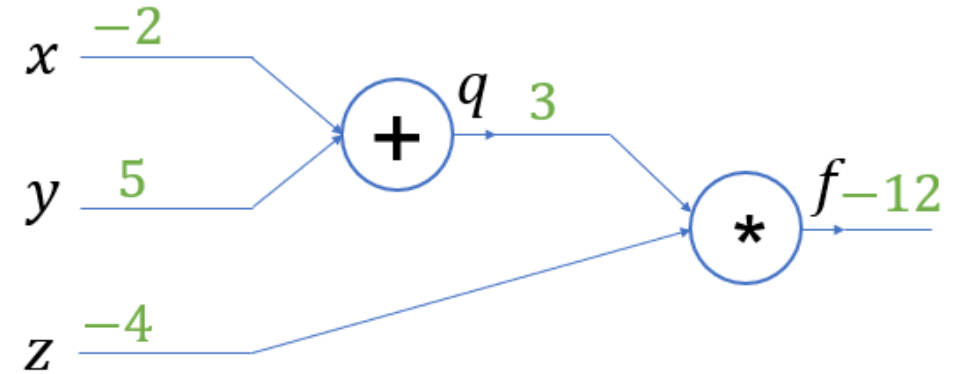
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z \quad \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: Simple example

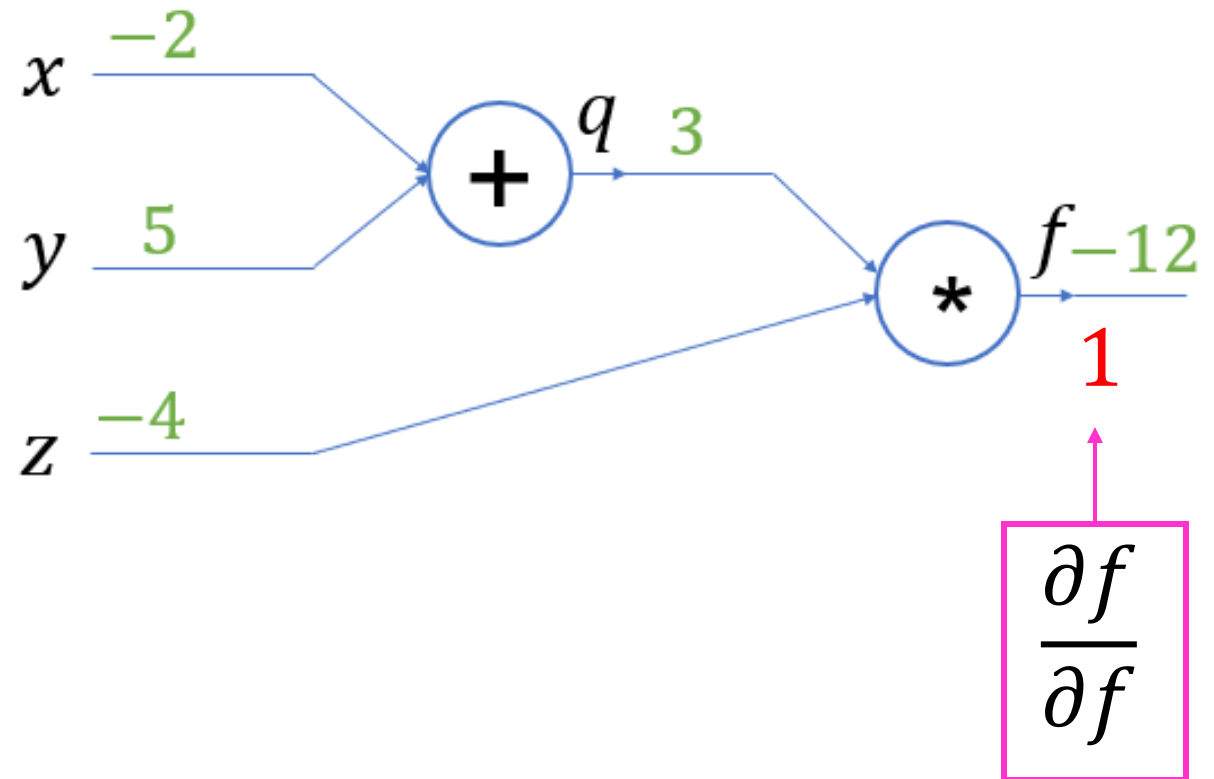
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z \quad \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: Simple example

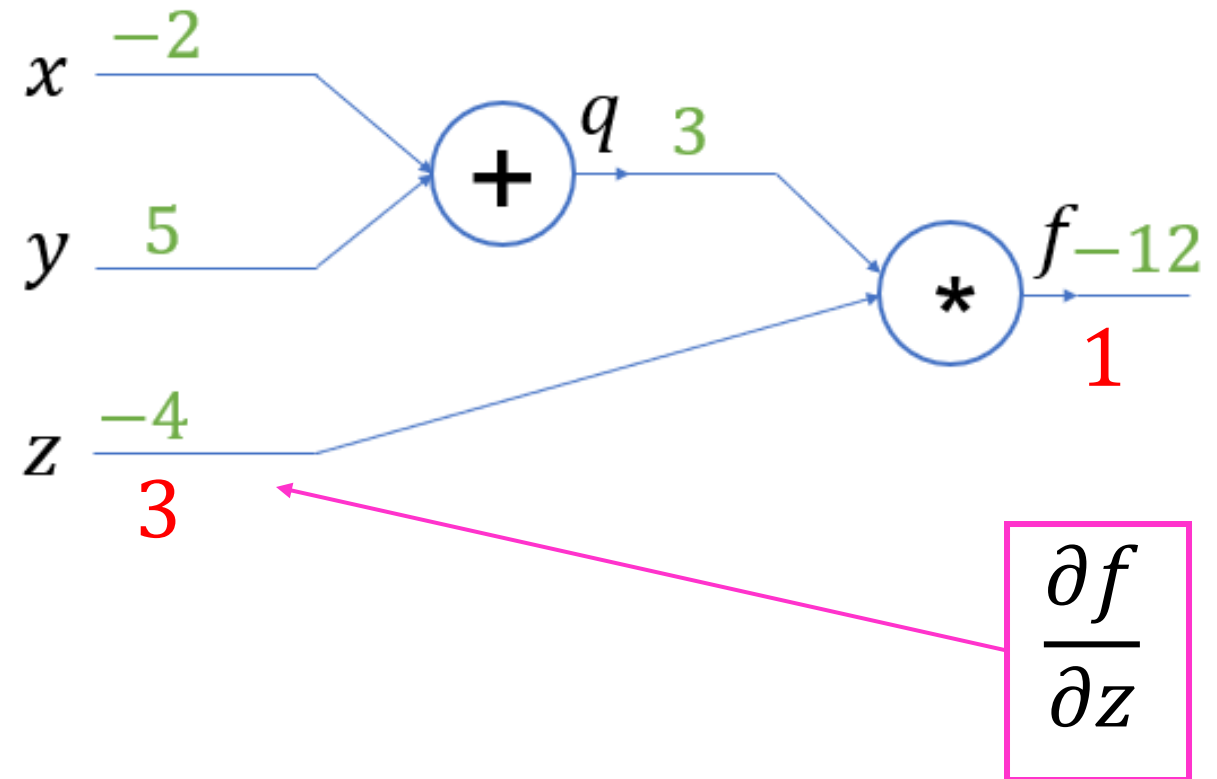
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z \quad \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Backpropagation: Simple example

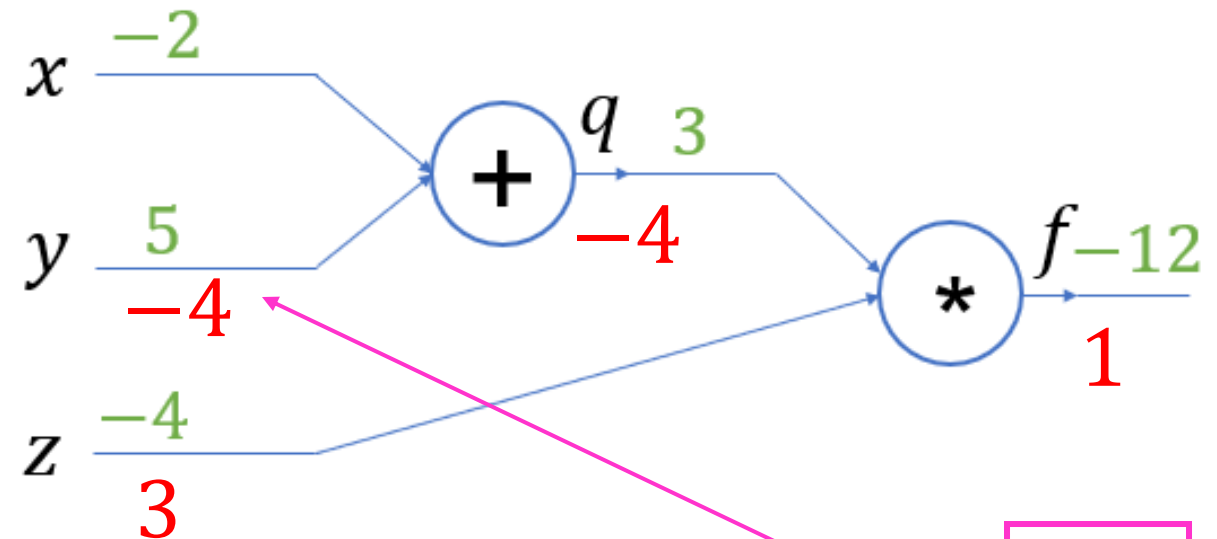
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z \quad \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

$$\frac{\partial f}{\partial y}$$

Backpropagation: Simple example

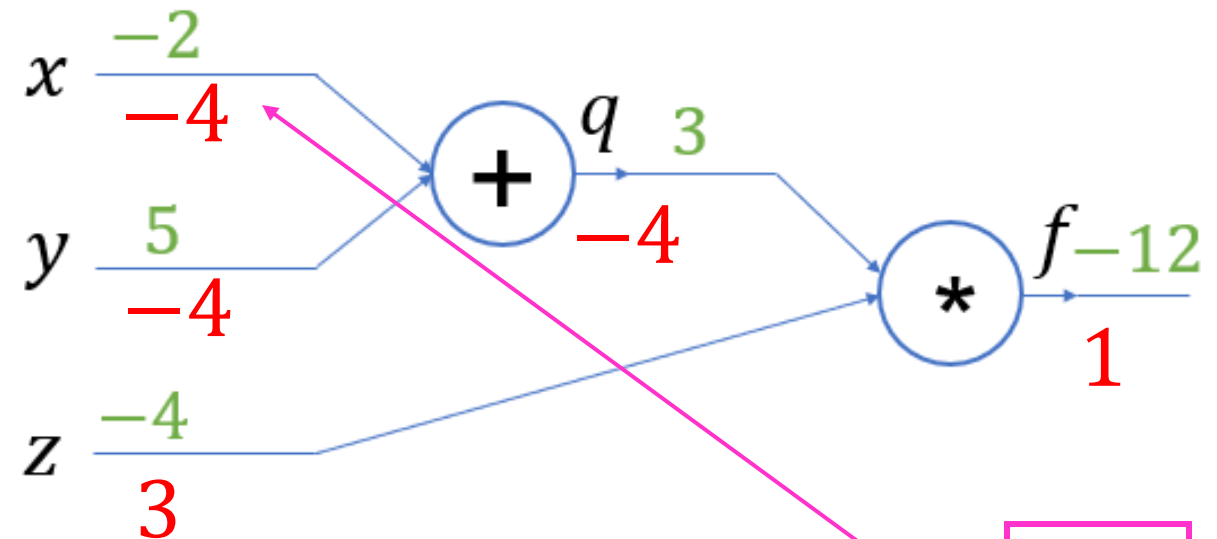
$$f(x, y, z) = (x + y)z$$

$$x = -2, y = 5, z = -4$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z \quad \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream gradient

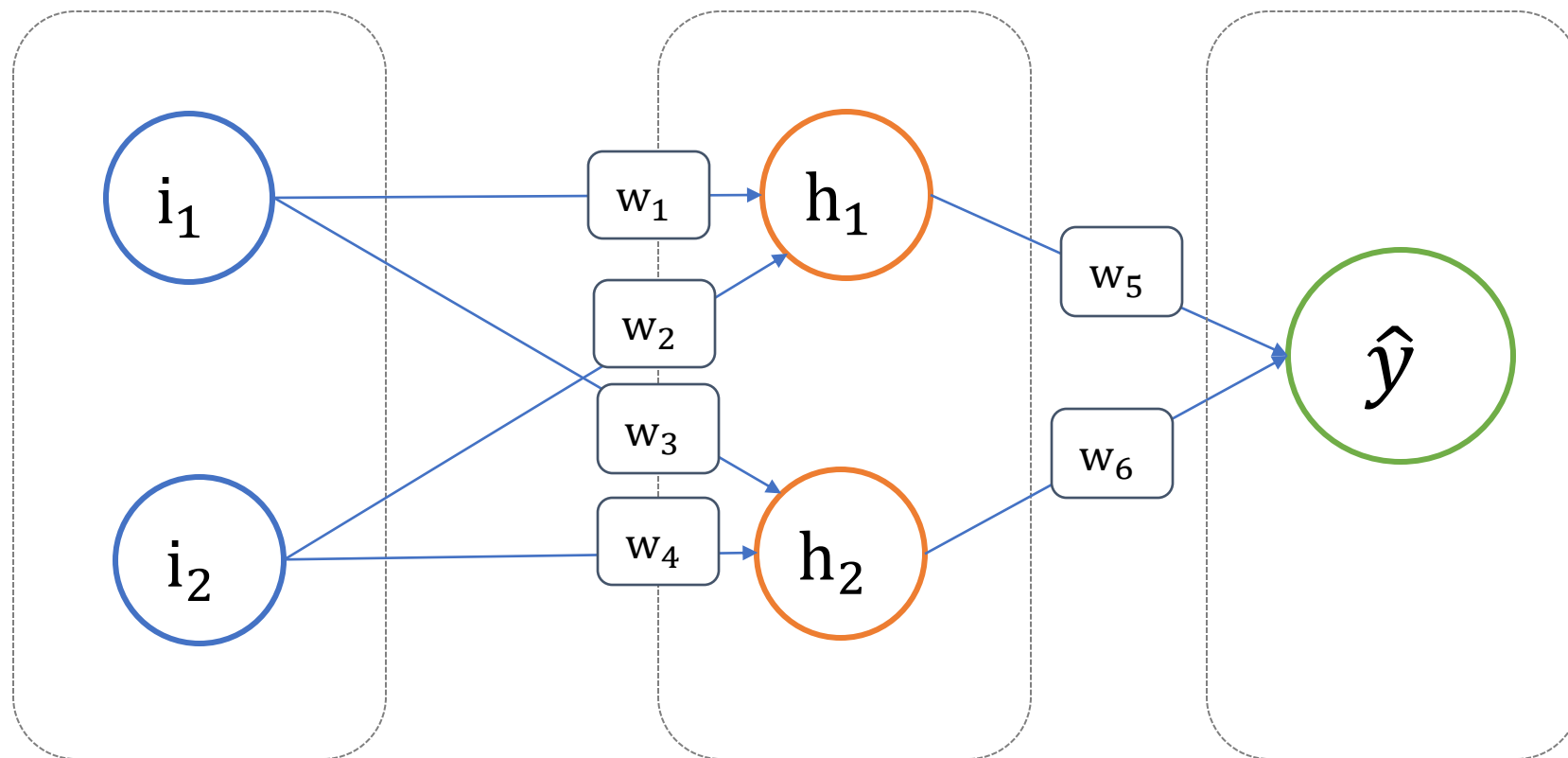
Local gradient

$$\frac{\partial f}{\partial x}$$

Input Layer

Hidden Layer

Output Layer



Backpropagation

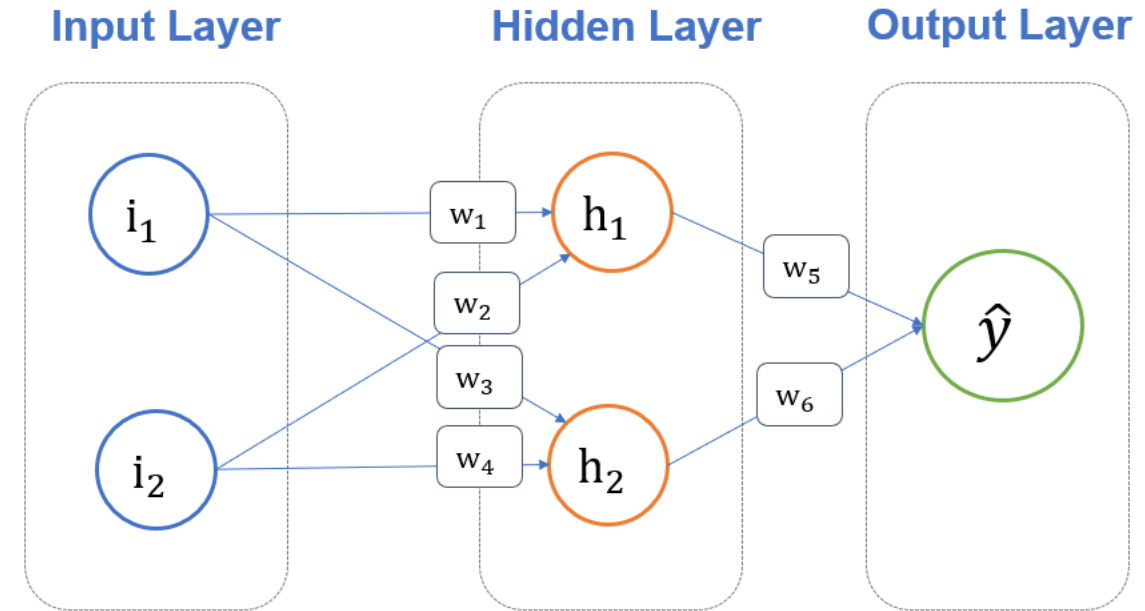
$$w_6 = w_6 - \alpha \left(\frac{\partial \text{Error}}{\partial w_6} \right)$$

$$\frac{\partial \text{Error}}{\partial w_6} = \frac{\partial \text{Error}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_6}$$

$$\frac{\partial \text{Error}}{\partial w_6} = \frac{1}{2} (y - \hat{y})^2 \cdot \frac{(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial w_6}$$

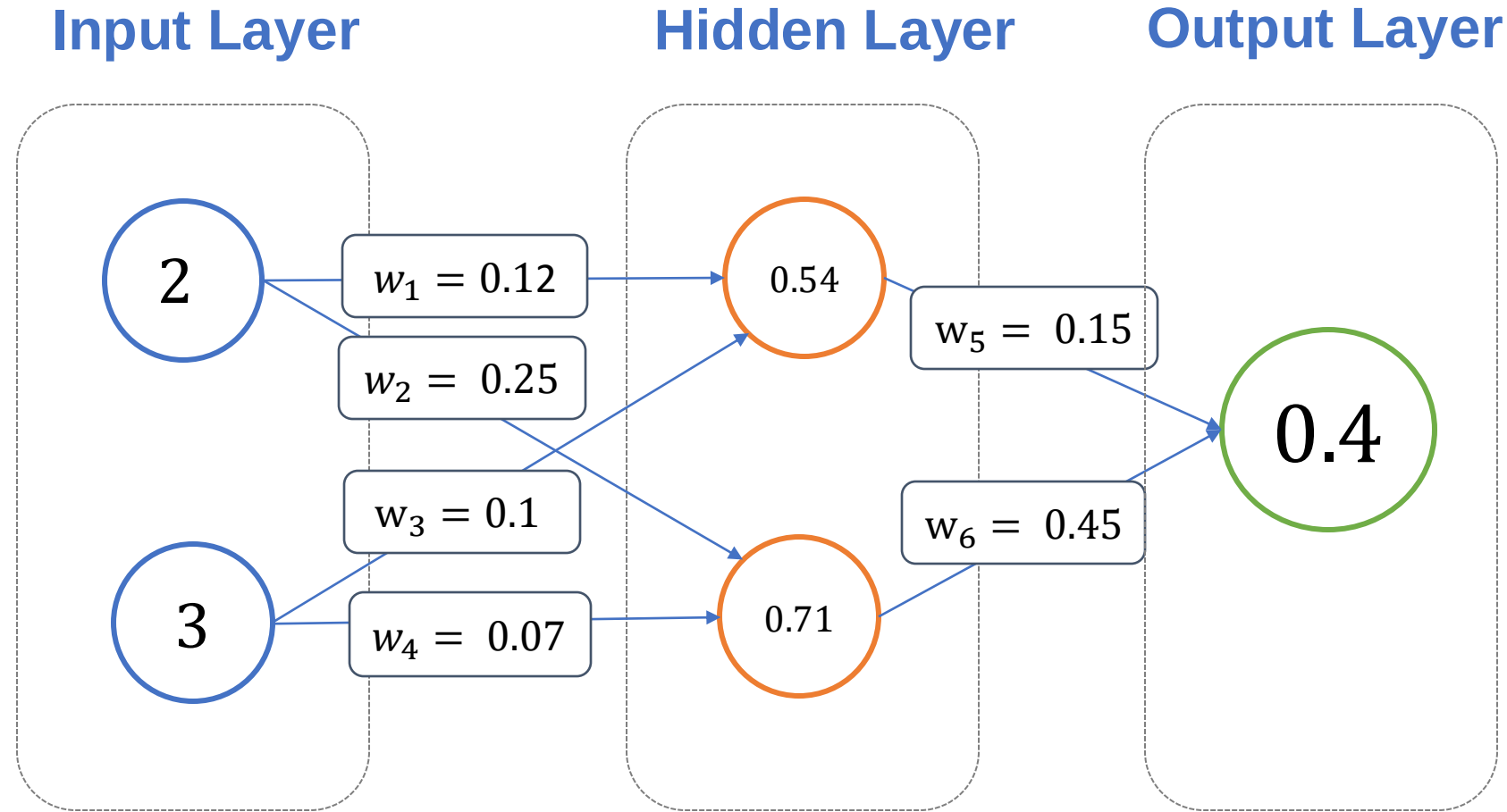
$$\frac{\partial \text{Error}}{\partial w_6} = 2 \cdot \frac{1}{2} (y - \hat{y}) \frac{(y - \hat{y})}{\partial \hat{y}} \cdot (i_1 w_3 + i_2 w_4)$$

$$\frac{\partial \text{Error}}{\partial w_6} = (y - \hat{y}) \cdot (-1)(h_2)$$



Backpropagation

$$w_5 = w_5 - \alpha(-h_1(y - \hat{y}))$$
$$w_6 = w_6 - \alpha(-h_2(y - \hat{y}))$$

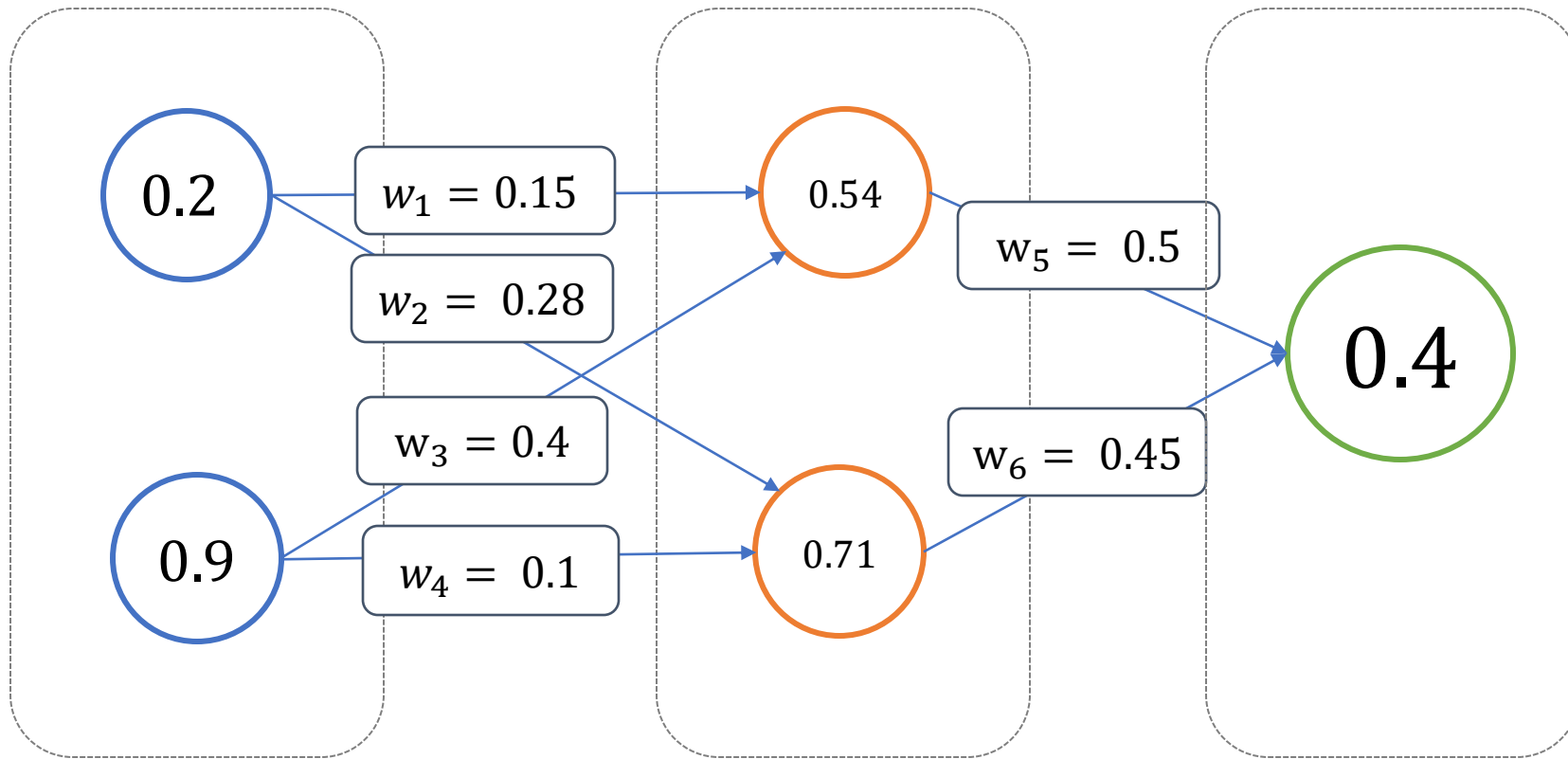


$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} =$$

Input Layer

Hidden Layer

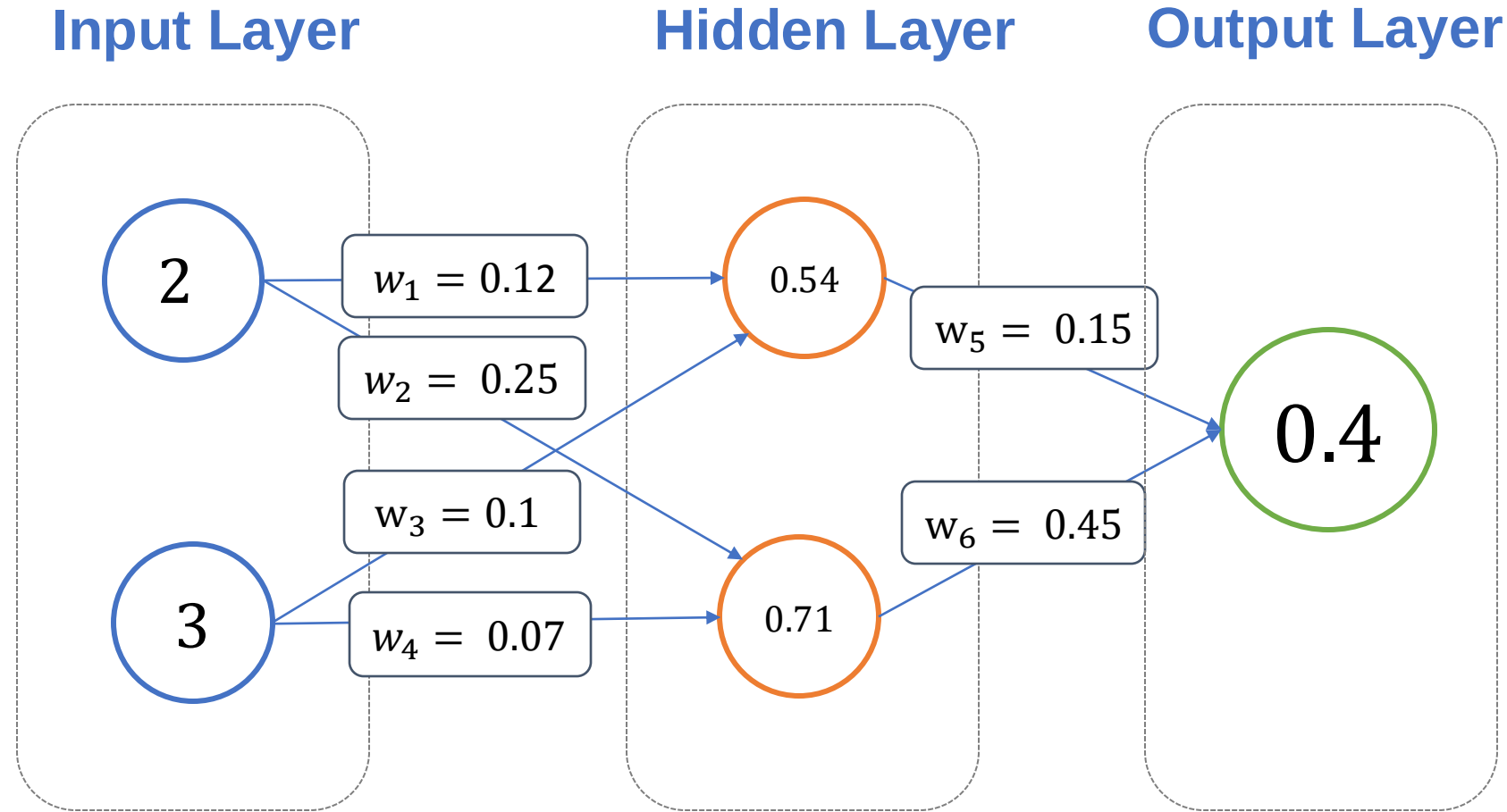
Output Layer



$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} =$$

Backpropagation

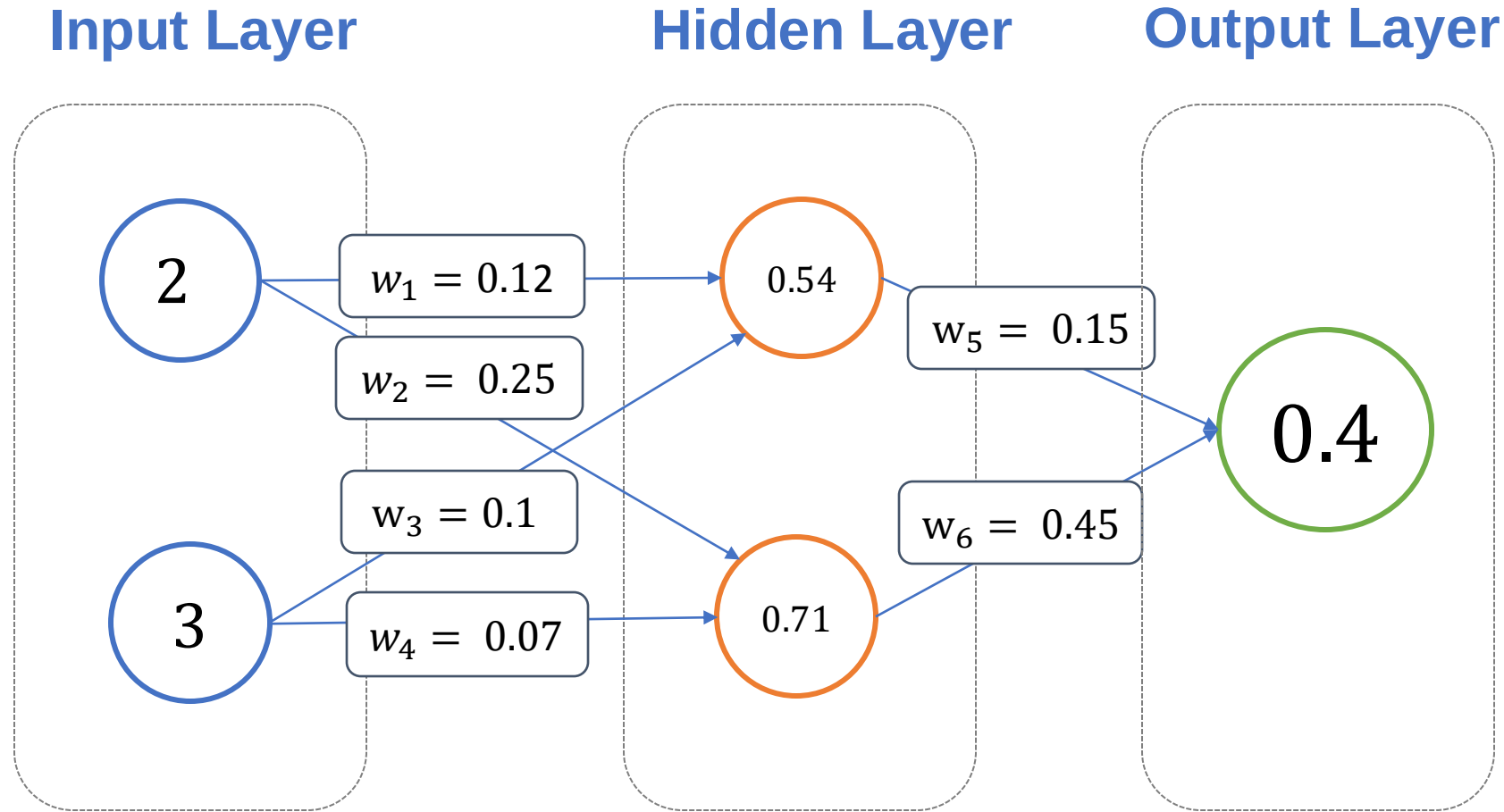
$$w_5 = w_5 - \alpha(-(y - \hat{y})h_1)$$
$$w_6 = w_6 - \alpha(-(y - \hat{y})h_2)$$



$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix} - 0.1 * -(0.6) \begin{bmatrix} 0.54 \\ 0.71 \end{bmatrix}$$

Backpropagation

$$w_5 = w_5 - \alpha(-h_1(y - \hat{y}))$$
$$w_6 = w_6 - \alpha(-h_2(y - \hat{y}))$$



$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix} - 0.1 * -(0.6) \begin{bmatrix} 0.54 \\ 0.71 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.45 \end{bmatrix} - \begin{bmatrix} -0.0324 \\ -0.0426 \end{bmatrix} = \begin{bmatrix} 0.18 \\ 0.49 \end{bmatrix}$$

Backpropagation

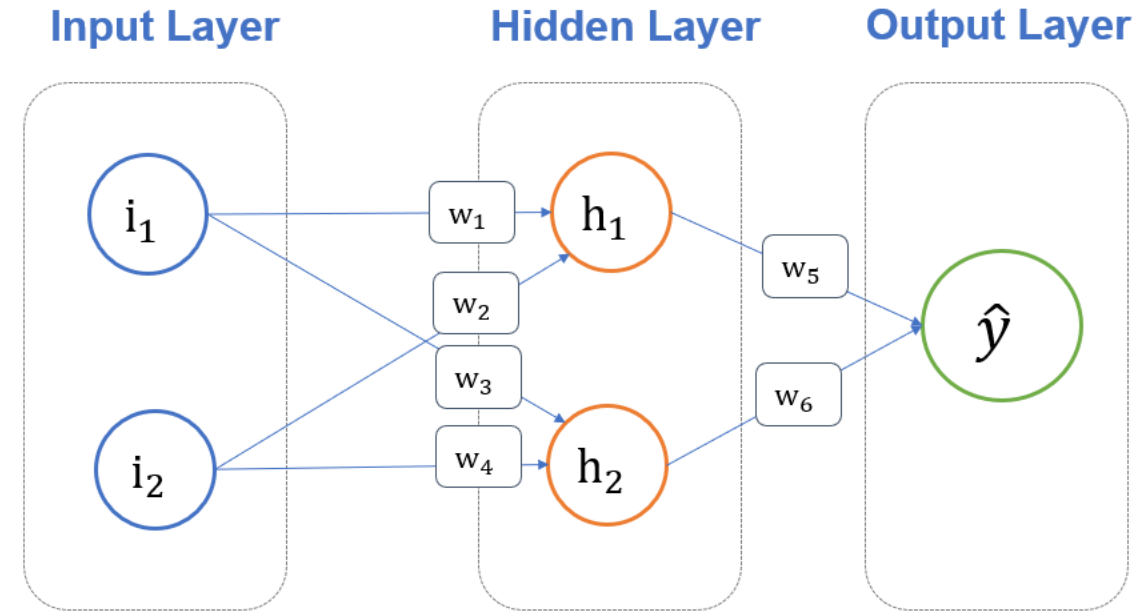
$$w_1 = w_1 - \alpha \left(\frac{\partial \text{Error}}{\partial w_1} \right)$$

$$\frac{\partial \text{Error}}{\partial w_1} = \frac{\partial \text{Error}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial w_1} = \frac{1}{2} (y - \hat{y})^2 \cdot \frac{\partial (h_1)w_5 + (h_2)w_6}{\partial h_1} \cdot \frac{\partial i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial w_1} = 2 \cdot \frac{1}{2} (y - \hat{y}) \cdot \frac{\partial (y - \hat{y})}{\partial \hat{y}} \cdot (w_5 * i_1)$$

$$\frac{\partial \text{Error}}{\partial w_1} = (y - \hat{y}) \cdot (-1) \cdot (w_5 i_1)$$



Backpropagation

$$w_6 = w_6 - \alpha(-h_2(y - \hat{y}))$$

$$w_5 = w_5 - \alpha(-h_1(y - \hat{y}))$$

$$w_4 = w_4 - \alpha(-i_2(y - \hat{y})w_6)$$

$$w_3 = w_3 - \alpha(-i_1(y - \hat{y})w_6)$$

$$w_2 = w_2 - \alpha(-i_2(y - \hat{y})w_5)$$

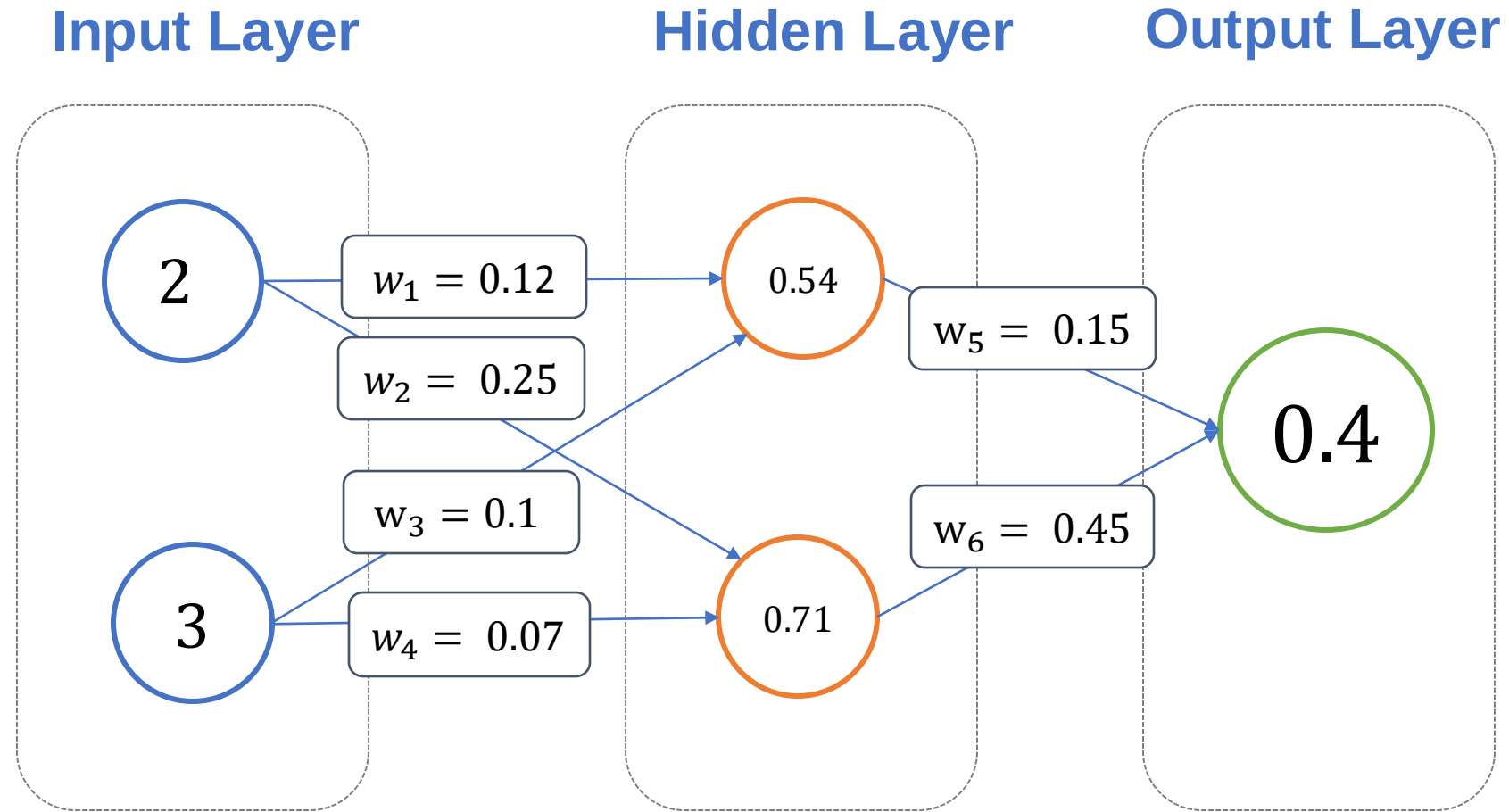
$$w_1 = w_1 - \alpha(-i_1(y - \hat{y})w_5)$$

Backpropagation

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \alpha(y - \hat{y})(-1) \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} \alpha h_1 \delta \\ \alpha h_2 \delta \end{bmatrix}$$

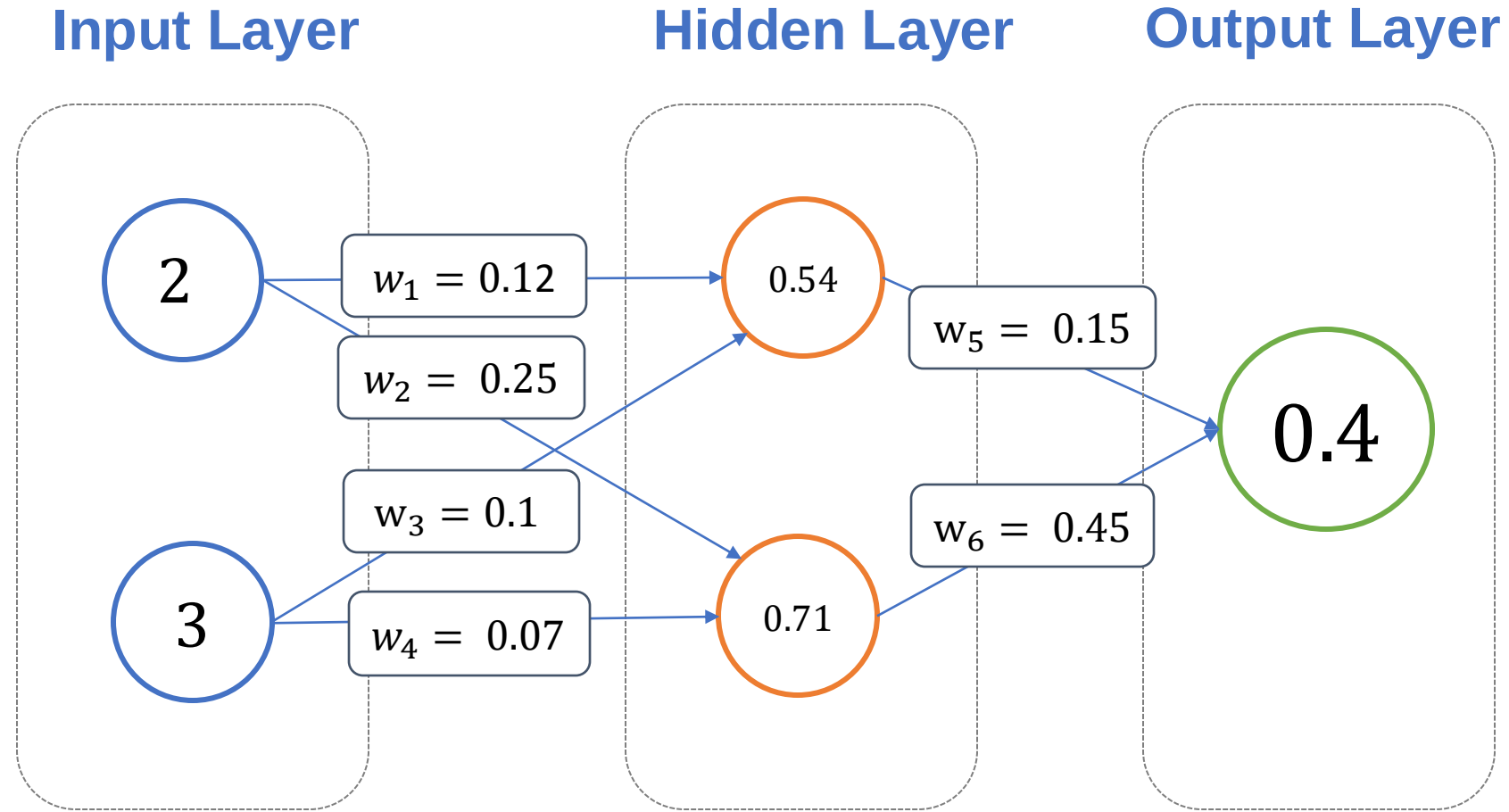
$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \alpha(y - \hat{y})(-1) \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \begin{bmatrix} w_5 & w_6 \end{bmatrix} =$$
$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} + \begin{bmatrix} \alpha i_1 (y - \hat{y}) w_5 & \alpha i_1 (y - \hat{y}) w_6 \\ \alpha i_2 (y - \hat{y}) w_5 & \alpha i_2 (y - \hat{y}) w_6 \end{bmatrix}$$

Backpropagation



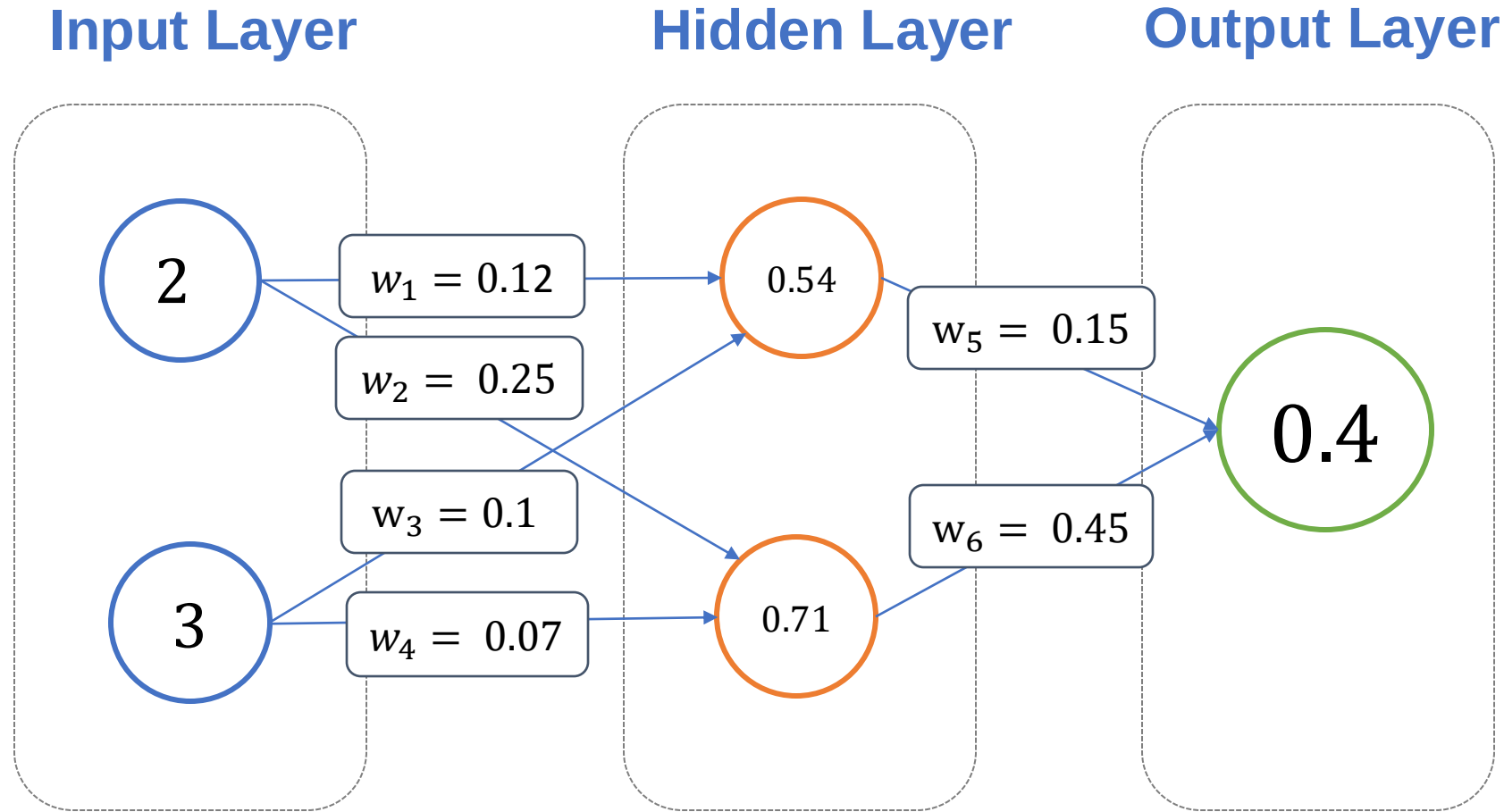
$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} =$$

Backpropagation



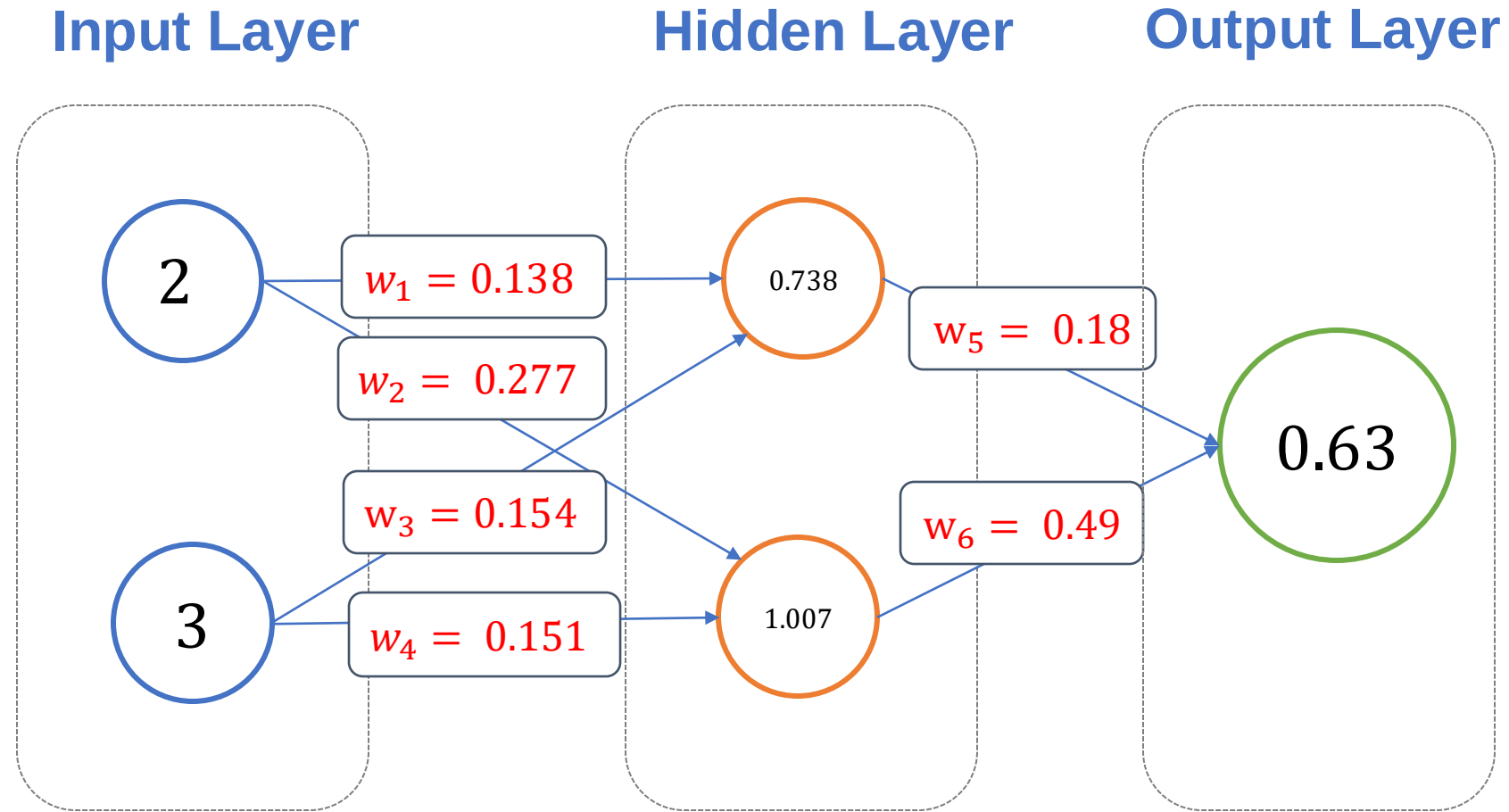
$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} 0.12 & 0.1 \\ 0.25 & 0.07 \end{bmatrix} - 0.1 * -(0.6) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} 0.15 & 0.45 \end{bmatrix} =$$

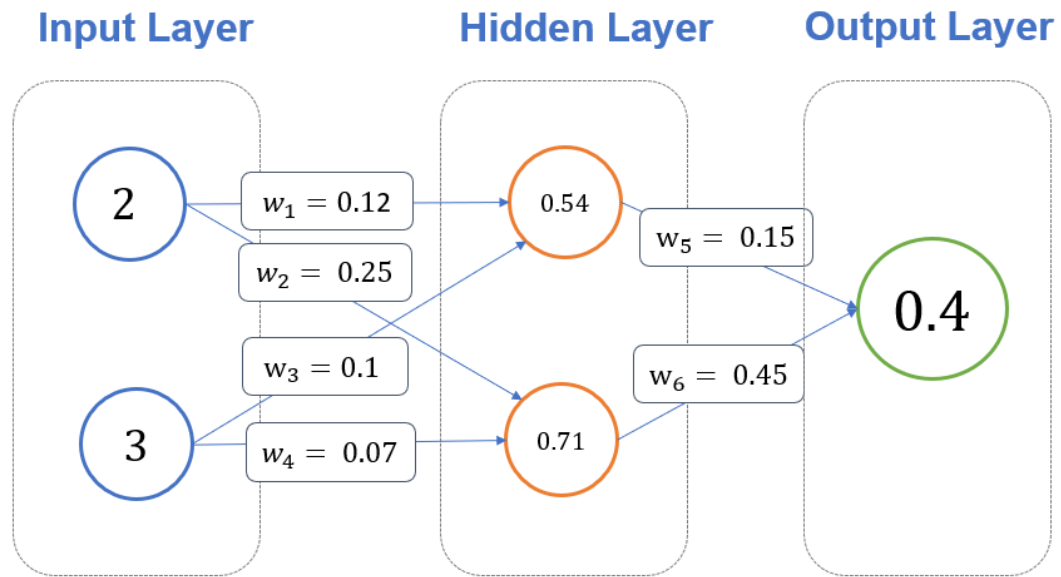
Backpropagation



$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} 0.12 & 0.1 \\ 0.25 & 0.07 \end{bmatrix} - 0.1 * -(0.6) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \begin{bmatrix} 0.15 & 0.45 \end{bmatrix} = \begin{bmatrix} 0.12 & 0.1 \\ 0.25 & 0.07 \end{bmatrix} - \begin{bmatrix} -0.018 & -0.054 \\ -0.027 & -0.081 \end{bmatrix} = \begin{bmatrix} 0.138 & 0.154 \\ 0.277 & 0.151 \end{bmatrix}$$

Backpropagation

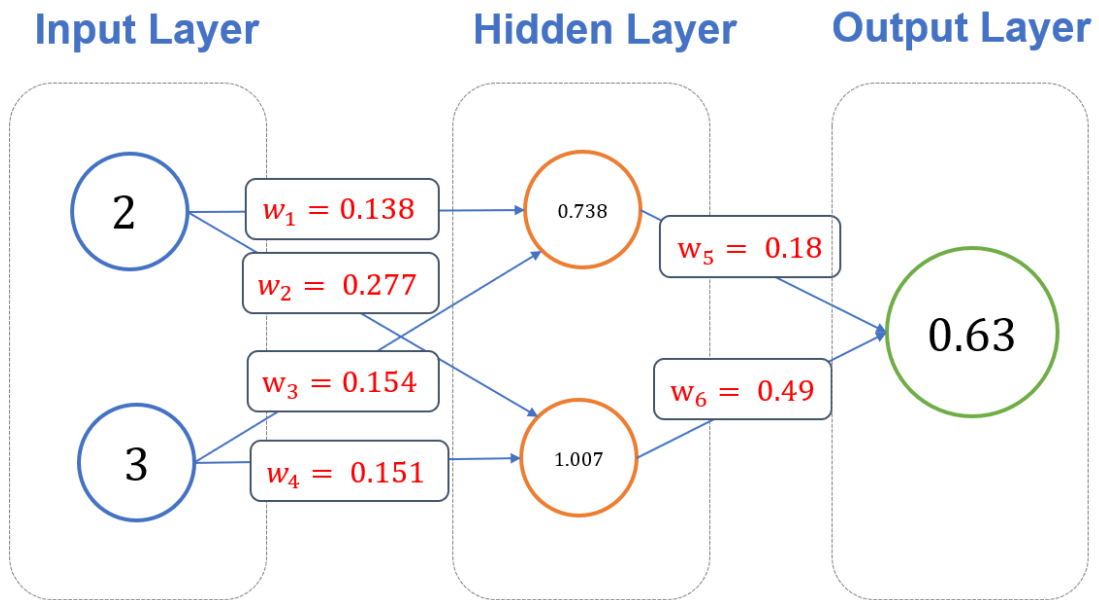


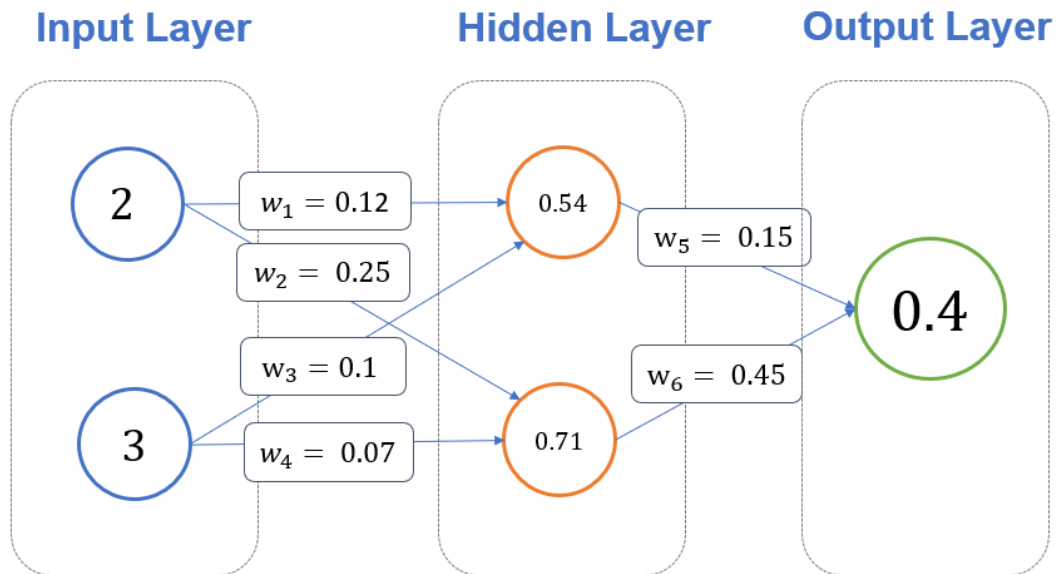


Target (y)

1

Error₁ =

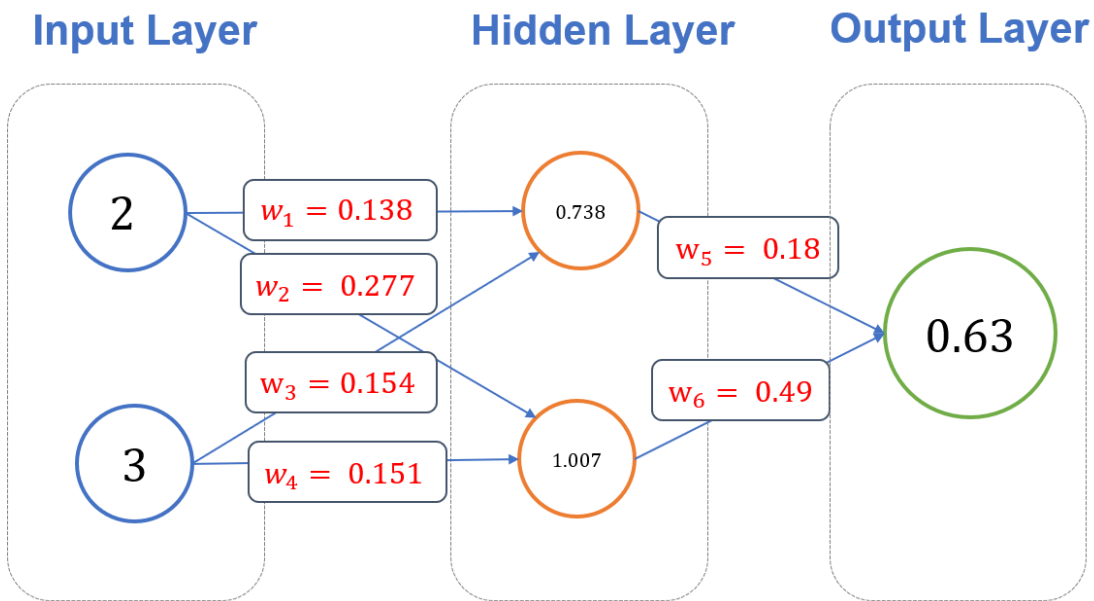




Target (y)

1

$$\text{Error}_1 = \frac{1}{2} (1 - 0.4)^2 = 0.18$$



$$\text{Error}_2 = \frac{1}{2} (1 - 0.63)^2 = 0.07$$

Summary so far

- Error function contains many local minima
- No guarantee of convergence
 - Not a “big” issue in practical deployments
- Improving backpropagation
 - Adding momentum
 - Using stochastic gradient descent
 - Train multiple times using different initializations